



UNIVERSITI TUNKU ABDUL RAHMAN (UTAR)

Lee Kong Chian Faculty of Engineering & Science (FES LKC)

**Bachelor of (Honours) Software Engineering [SE]/ Applied
Mathematics with Computing [AM]**

Project Survive till Last

Group Name	Survivor
Student Names & (IDs)	Angel Yap Yoon Ying 2205499 Huang Wan Jun 2204536 Ng Poh Hui 2204747 Tiu Han Xuen 2205501
Due Date	
Lecturer	Tai Kwang Liang
Marks	

Table of Contents

1.0 Introduction	5
2.0 Preface	5
3.0 Student Declaration Page	6
4.0 Project Overview	6
4.1 Game Overview	7
4.2 Game Premise	8
4.3 Prominent Game Features	8
4.4 Team Designation	10
5.0 Game Engine Details	12
5.1 Game Engine	13
5.2 Game Engine Features	13
5.3 Scripting/Programming Language	15
5.4 Game Engine Technical Issues and Setup	16
5.5 Hardware Technical Specifications and Requirements	17
6.0 Project Technical Design Overview	18
6.1 Game Technical Specifications	18
6.2 Project Game Development Setup	19
6.3 Project Organization Overview	20
6.4 Game Flow / Scenes Overview	22
7.0 Programming Conventions and Designs	23
7.1 Overall Script Setup	24
7.2 Coding Conventions and Organization	25
7.3 Naming Conventions	26
7.3.1 Script and File Naming	26
7.3.2 Class Naming	26

7.3.3 Variable and Function Naming.....	27
7.3.4 Signal Naming.....	28
7.3.5 Summary of Naming Conventions.....	28
7.4 Overall Design Pattern Conventions.....	28
8.0 Class Diagrams/UML Overview	29
8.1 System UML Class Diagram	29
8.2 Class List and Overview	29
8.3 Methods Overview by Class	30
8.4 Variable Overview by Class	31
8.5 Feature Implementation Summary	32
8.6 Game Object Design Setup.....	33
8.7 Player Controls Implementation Overview	33
9.0 Audio Implementation Overview.....	35
9.1 Audio File Setup & Bus Hierarchy.....	35
9.2 Sound Effects Overview	36
9.3 Background Track Overview	37
10.0 Game AI Overview.....	37
10.1 Game AI Overview & State Machine Script Architecture	37
10.2 Enemy Type 1: Slime (slime.gd).....	38
10.3 Enemy Type 2: Skeleton (skeleton.gd)	38
10.4 Enemy Type 3: Ranged Enemy (RangedEnemy.gd)	39
10.5 Enemy Type 4: Final Boss (final_Boss.gd).....	40
10.6 Weapon Type	42
10.6.1 Weapon 1: Knife.....	42
10.6.2 Weapon 2: Gun.....	47
10.6.3 Weapon 3: Guard Halo.....	51

10.6.4 Weapon 4 : Chain Lightning	56
10.6.5 Weapon 5: Gravity Bomb	59
10.6.6 Weapon 6: Temporal Echo	64
10.6.7 Weapon 7: Astral Prism Snare	69
11.0 Interactable Objects & GUI Systems	74
11.1 Interactable Objects Design & Implementation	74
11.2 Game Interface Setup and Implementation (GUIs)	74
11.3 Camera Setup and Implementation	75
11.4 Canvas and other GUI Setups	76
11.5 Game Options and Implementation	76
12.0 Project Development Progression	77
12.1 Project Schedule	77
References	78

CHAPTER 11.0 Introduction

Survive till Last is a 2.5D action survival game developed by the Survivor group. The game combines the enemy wave survival concept of Vampire Survivors with the dungeon-based combat and weapon mechanics inspired by Soul Knight (Poncle, n.d.; ChillyRoom, n.d.). The main objective of the game is for the player to survive against continuous waves of enemies within a dungeon environment while collecting experience shards, levelling up and obtaining stronger weapons.

The game uses a 2.5D visual style. The dungeon environment is constructed using 3D elements. The player character, weapons and enemies are represented using 2D pixel art sprites. This combination provides a three-dimensional gameplay environment while maintaining a pixel art visual appearance.

During gameplay, players are required to manage their health, weapons and character progression while dealing with different types of enemies and environmental hazards. The game includes features like a health and experience system, multiple weapons, a six-slot weapon inventory, enemy waves, level up choices and a boss encounter.

This technical document describes the design and implementation of Survive till Last. It provides an overview of the game engine, project structure, programming implementation, game mechanics, player and enemy systems, user interface, physics, animation and development progression.

CHAPTER 22.0 Preface

This technical document presents the development and technical implementation of Survive till Last. It is a 2.5D action survival game developed by the Survivor group. The document explains the main technical components of the game. This includes the game engine, project structure, programming implementation, player controls, weapons, enemy systems, game mechanics, user interface, animation, physics and overall development process.

The purpose of this document is to provide a clear reference to how the game was designed and implemented. It also documents the technical decisions and development approaches used by the team throughout the project.

The document is prepared as part of the game development project and is intended to demonstrate the technical work completed by the team in developing Survive till Last.

CHAPTER 33.0 Student Declaration Page

This technical document presents the development and technical implementation of Survive till Last. It is a 2.5D action survival game developed by the Survivor group. The document explains the main technical components of the game. This includes the game engine, project structure, programming implementation, player controls, weapons, enemy systems, game mechanics, user interface, animation, physics and overall development process.

CHAPTER 44.0 Project Overview

Survive till Last is a 2.5D action survival game developed by the Survivor group. The project focuses on survival-based dungeon combat where the player must survive enemy waves while collecting experience shards to increase their level and obtaining stronger weapons. The game combines the enemy-wave survival concept of Vampire Survivors with the dungeon combat and weapon style inspired by Soul Knight (Poncle, n.d.; ChillyRoom, n.d.).

The game uses a 2.5D visual style by combining a 3D dungeon environment with 2D pixel art sprites. The dungeon, obstacles and surrounding environment are presented using 3D elements while the player character, enemies and weapons use 2D sprites. This approach allows the game to provide a three-dimensional gameplay environment while maintaining a pixel art visual style.

The main gameplay involves controlling the player within the dungeon while facing different types of enemies and environmental hazards. Players can use different weapons, collect experience shards and level up during a run. The game includes a six-slot weapon inventory, a health system, weapon upgrades and a level up system that provides three upgrade choices to the player.

The project also includes different enemy behaviours, enemy waves and a boss encounter. These features are designed to progressively increase the challenge and require the player to manage movement, health and available weapons throughout the game.

Overall, Survive till Last is designed to provide a fast-paced survival experience that combines dungeon combat, enemy waves, weapon progression and character development within a 2.5D game environment.

4.1 Game Overview

Survive till Last is a 2.5D action survival game where the player controls a character within a dungeon environment and attempts to survive against groups of enemies. The gameplay is inspired by the enemy-wave concept found in Vampire Survivors (Poncle, n.d.).

At the beginning of a game session, the player controls the character within the dungeon and uses the available weapon to defeat enemies. As enemies are defeated, experience shards can be collected to increase the player's experience and level. When the player levels up, the game provides three upgrade choices. This allows the player to improve their character or weapons during the run.

As the game progresses, the player encounters different enemy behaviours and increasingly challenging encounters. The player must manage their health, movement and available weapons while navigating the dungeon and avoiding environmental hazards. The game also includes a boss encounter as part of the progression.

The game uses a combination of 3D environmental elements and 2D pixel art sprites. This creates the 2.5D presentation while allowing the player, enemies and weapons to remain visually consistent with the game's pixel art style.

Overall, the gameplay loop consists of moving through the dungeon, defeating enemies, collecting experience shards, levelling up, selecting upgrades and surviving increasingly challenging encounters until the player is defeated or reaches the later stages of the game.

4.2 Game Premise

Survive till Last is set within a dangerous dungeon where the player is required to survive against continuous enemy encounters. The player enters the dungeon and must fight through groups of enemies while attempting to remain alive for as long as possible.

During the run, enemies appear within the dungeon and challenge the player through different movement and attack behaviours. The player must move around the environment, use available weapons and avoid environmental hazards while dealing with the increasing threats.

Defeating enemies allows the player to collect experience shards which contribute towards levelling up. When the player reaches a new level, three upgrade choices are presented. This is to allow the player to strengthen their character or weapons. This progression allows the player to become more capable as the difficulty of the encounters increases.

The game progresses through different enemy encounters and eventually leads to a boss encounter. Overall, the premise is centred on surviving the dungeon, defeating enemies, improving the player's abilities and reaching the later stages of the game without losing all health.

4.3 Prominent Game Features

Survive till Last includes several gameplay and interface features that support the player's survival and progression throughout the game.

- Player Health System
 - The player has a health system that decreases when taking damage from enemies or environmental hazards. The game ends when the player's health reaches zero.
- Weapon System
 - The game provides different weapons that allow the player to attack enemies. Each weapon has its own attack behaviour and can be upgraded to improve its effectiveness.
- Six slot Weapon Inventory
 - The player can equip and manage up to six weapons during a run. The inventory allows different weapons to be stored and selected as the player progresses. The project also provides input controls for the six weapon slots.
- Experience and Level Up System
 - Enemies can be defeated to obtain experience shards. Collecting enough experience allows the player to level up and select an available upgrade to improve the player's abilities or weapons.
 - During level progression, the player can select an available upgrade that affect different attributes like health, experience progression and movement speed.
- Enemy Wave System
 - Enemies appear in waves and have different behaviours. The player must survive these encounters while managing their health, movement and weapons.
- Environmental Hazards
 - The game includes environmental hazards like spike traps and fire-based hazards. These hazards increase the difficulty of certain encounters and require the player to avoid dangerous areas while fighting enemies.

- **Boss Encounter**
 - A boss encounter is included as a later challenge in the game. Before reaching the boss stage, the player must complete multiple enemy waves and deal with additional environmental hazards.
- **2.5D Game Environment**
 - The game combines a 3D dungeon environment with 2D pixel art sprites for the player, enemies and weapons. This creates the game's 2.5D visual presentation.
- **Game Interface**
 - The game includes a main menu, gameplay HUD, weapon inventory display, level up selection interface and game over interface. The main menu also provides access to game functions like starting the game, viewing permanent upgrades and quitting the game.
- **Dungeon Mini Map**
 - The game includes a mini map during gameplay to help the player understand the surrounding dungeon layout and identify relevant entities. The mini map is implemented as a dedicated DungeonMinimap system within the dungeon interface.

4.4 Team Designation

The development of Survive till Last is divided among four team members. Each of the member responsible for different aspects of the game development. The main responsibilities of each team member are summarised in Table 4.1.

Team Member	Student ID	Main Responsibility
Angel Yap Yoon Ying	2205499	Weapon and Inventory System
Huang Wan Jun	2204536	Dungeon Map, Level Progression, Enemy Spawning, traps and Mini map
Ng Poh Hui	2204747	Player Controller and Level Up System

Tiu Han Xuen	2205501	Enemy AI and Wave System
--------------	---------	--------------------------

Table 4.1: Team Designation

Angel Yap Yoon Ying – Weapon and Inventory System

Angel Yap Yoon Ying is responsible for implementing the weapon and inventory systems. Her work includes weapon attack scripts, weapon selection, projectile behaviour and projectile trajectories. She also implemented the six-slot weapon inventory that allows the player to manage and select different weapons during gameplay. In addition, her responsibilities include combat feedback features like screen shake, damage pop effects and floating damage numbers.

Huang Wan Jun – Dungeon Map, Level Progression, Enemy Spawning, Traps and Mini Map

Huang Wan Jun is responsible for designing and implementing the dungeon environments and level layouts. Her work includes the 3D dungeon grid map, spatial layout, lighting, collision boundaries and static obstacles. She also implemented the enemy spawning and wave setup for each level. This includes displaying the number of remaining enemies during gameplay.

In addition, she implemented the level clear system which displays a Level Clear indication after all enemies in the current level have been defeated. The level progression system also includes blocking mechanisms that prevent the player from entering the next level before the current level has been completed. She also developed environmental traps for different level events and implemented the dungeon mini map to help players understand the current dungeon layout and relevant entities.

Ng Poh Hui – Player Controller and Level Up System

Ng Poh Hui is responsible for implementing the player controller and player movement system. Her work includes directional movement using keyboard

controls and the implementation of the player's directional animations. She also worked on the experience collection and level up system. This includes the level up selection interface and player stat progression. These systems allow the player to improve their abilities as they gain experience during gameplay.

Tiu Han Xuen – Enemy AI And Wave System

Tiu Han Xuen is responsible for implementing the enemy AI and enemy behaviours. Her work includes enemy movement, vector-based pathfinding and different enemy behaviours. She also worked on the experience-share attraction system and the time-based enemy wave system. In addition, she is responsible for the boss encounter system. This includes the different enemy waves and boss related encounter behaviours.

CHAPTER 55.0 Game Engine Details

Survive till Last is developed using the Godot Engine as the main game development environment. Godot is used to create and manage the game's scenes, gameplay systems, user interfaces, character controls, enemy systems, weapons and 2.5D game environment.

The project is developed using Godot 4.6.3 with the Forward Plus rendering method. The project configuration also uses Jolt Physics for 3D physics. The main game scene is configured as MainMenu.tscn which serves as the starting scene when the game is launched.

The game combines 3D environmental elements with 2D game sprites to achieve its 2.5D presentation. Godot is used to manage different gameplay systems. This includes the player, enemies, weapons, dungeon levels, mini map, menus, progression systems and user interfaces.

The following subsections describe the game engine, its features, programming language and technical setup used in the development of Survive till Last.

5.1 Game Engine

The game is developed using the Godot Engine which serves as the main environment for creating and managing the game's scenes, objects, scripts, gameplay systems and user interfaces.

The project uses Godot 4.6.3 with the Forward Plus rendering method and Jolt Physics for 3D physics processing. These configurations are defined in the project's project.godot file.

Godot is used to develop the different components of Survive till Last. This includes the 3D dungeon environment, 2D player and enemy sprites, weapons, enemy systems, level progression, environmental hazards, mini map and game interfaces. The project separates different functions into scenes and scripts to make the game systems easier to manage and develop.

The main game scene is configured as `res://UI/MainMenu.tscn`. This makes the Main Menu as the starting point when the game is launched.

5.2 Game Engine Features

The Godot Engine provides several features that support the development of Survive till Last. The main features used in the project are described below.

- **Scene-Based Development**
 - Godot uses a scene-based structure to organise different parts of the game. Survive till Last uses separate scenes to manage the components. These components are main menu, dungeon levels, player, enemies, weapons, mini map and user interfaces.
- **2D and 3D Development**

- Godot supports both 2D and 3D development. This allows the game to combine a 3D dungeon environment with 2D pixel art sprites. This combination is used to create the game's 2.5D presentation.
- **GDScript Support**
 - GDScript is used to implement the game's gameplay logic and systems. Scripts are attached to relevant nodes and scenes to control player movement, enemy behaviour, weapons, level progression, spawning, hazards and interface interactions.
- **Physics System**
 - Godot provides physics and collision functionality for handling interactions between game objects. The project is configured to use Jolt Physics for its 3D physics processing.
- **User Interface System**
 - Godot's Control nodes and UI components are used to create the game's different interfaces. These include the Main Menu, gameplay HUD, weapon inventory, level up interface, game over interface and other gameplay information.
- **Input System**
 - Godot's input system is used to handle player controls and weapon selection. The project contains input actions for directional movement, attacking and selecting the six weapon slots.
- **Main Menu System**
 - The game includes a Main Menu that provides access to the main game functions. The menu contains options such as Start Game, Upgrades and Quit. It also includes profile related functionality and a permanent upgrade interface.
- **Pause Menu System**
 - The project includes a pause menu system that allows gameplay to be controlled through a dedicated pause system. The PauseMenu system is configured as an autoload. So, it can be accessed across different scenes.

- **Audio System**

- The project includes an audio management system through the AudioManager autoload. This allows audio related functions to be managed across the game and shared between different scenes.

- **Save and Profile System**

- The project includes a SaveSystem autoload for managing persistent game related data. The Main Menu also contains profile functionality that allows a player profile to be selected or created.

- **Weapon Progression System**

- The project includes a WeaponProgress autoload to manage weapon related progression across scenes. This supports the game's weapon progression and upgrade functionality.

- **Mini Map System**

- The game includes a dedicated DungeonMinimap system that is integrated into the dungeon interface. The minimap provides information about the dungeon environment and relevant entities during gameplay.

Overall, these Godot features allow our team to organise the game into separate systems. It also supports the game's 2.5D environment, gameplay mechanics, audio, menus, player progression, enemy encounters and user interfaces.

5.3 Scripting/Programming Language

The main scripting language used to develop Survive till Last is GDScript. GDScript is used within the Godot Engine to implement the game's gameplay logic and control the behaviour of different game objects.

The scripts are used to implement important game systems. This includes player movement, player health, enemy behaviour, weapon systems, experience collection, level progression, enemy spawning, environmental hazards and user

interface interactions. The project also separates these functions into different scripts and scenes to organise the game's systems.

For instance, the player controller is implemented using a GDScript that handles directional movement, attacking, jumping, health, experience and level progression. The project also contains separate scripts for systems like the dungeon minimap and boss encounter controller.

Overall, GDScript provides the main programming interface for connecting the game's scenes. It implements the interactive gameplay mechanics of *Survive till Last*.

5.4 Game Engine Technical Issues and Setup

The development of *Survive till Last* requires several technical configurations within the Godot Engine to support the game's 2.5D environment and gameplay systems. The project is configured using Godot 4.6.3 with the Forward Plus rendering method and Jolt Physics for 3D physics processing.

The project also requires the appropriate configuration of input actions to support player movement, attacking and weapon selection. Six weapon input actions are configured to allow the player to select weapons from the six-slot weapon inventory.

Several global systems are configured using Godot's autoload functionality. These include `SaveSystem`, `WeaponProgress`, `PauseMenu` and `AudioManager`. The project uses autoloads. This is to allow these systems to remain accessible across different scenes without requiring each scene to create a separate instance.

The project also uses a custom cursor and custom font as part of the game's interface configuration. These settings are defined within the project configuration. They are applied to the game interface.

The team members needed to manage the interaction between the 3D dungeon environment and the game's 2D sprites during development. The project

also required the different gameplay systems. This includes the enemy spawning, level progression, weapons, mini map, user interface and player systems. These different gameplay systems are needed to work together across different scenes.

Overall, technical setup of the Godot project provides the required environment for developing and integrating the game's gameplay, visual, audio, input and progression systems.

5.5 Hardware Technical Specifications and Requirements

The development of Survive till Last was carried out using the personal laptops of the four team members. Each team member used their own laptop for developing, testing and running the game throughout the project. Therefore, the hardware specifications may differ between team members.

The laptops were used to run the Godot Engine for developing the game scripts and scenes, create the 2.5D dungeon environment and test the different gameplay systems. The hardware also needed to support the game's combination of 3D environmental elements and 2D pixel art sprites.

The general hardware components required for the development and testing of the game include a suitable processor, sufficient RAM, graphics processing capability and adequate storage space. A keyboard and mouse are also required for development and gameplay testing.

Hardware Component	Requirement or Usage
Computer	Personal laptop for each team member
Processor (CPU)	Capable of running Godot Engine and the game
Memory (RAM)	Sufficient memory for Godot and game development
Graphics (GPU)	Capable of rendering the game's 3D environment and 2D sprites

Storage	Sufficient space for Godot, project files and game assets
Input Device	Keyboard and mouse
Operating System	Windows
Number of Development Devices	Four personal laptops

The project is configured for Windows. Since each team member uses a different personal laptop, the exact CPU, GPU, RAM and storage specifications vary between members.

Team Member	CPU	GPU	RAM	Storage
Angel Yap Yoon Ying	Intel Core i5-1135G7	NVIDIA GeForce MX350	16.0 GB	512 GB
Huang Wan Jun	AMD Ryzen 5	NVIDIA GeForce RTX 4050	32.0 GB	512 GB
Ng Poh Hui	Intel Core i5-1235U	Intel(R) Iris(R) Xe Graphics Family	16.0 GB	512 GB
Tiu Han Xuen	Intel Core i5-12450H	NVIDIA GeForce RTX 3050	24.0 GB	477 GB

CHAPTER 66.0 Project Technical Design Overview

6.1 Game Technical Specifications

Survive-till Last is designed to provide a seamless high-performance gaming experience at a consistent frame rate of 60 FPS with vertical sync turned on by

default in order to avoid the screen tearing. The rendering architecture utilizes the Forward+ rendering engine of Godot 4.6, along with Direct3D 12 on Windows systems, that enables efficient 2D draw-call batching, hardware-accelerated particle simulation, and advanced lighting calculations.

In order to support different types of display hardware and monitor's aspect ratios, the display pipeline offers a scalable `canvas_items` stretch technique. This approach allows the game to keep its internal rendering at the same resolution and at the same time make GUI borders and camera viewports to dynamically adapt to higher widescreen resolutions without pixel distortion and aspect ratio changes. The game's spatial processing is based on a 2.5D isometric projection method that allows to convert the usual 2D Cartesian vectors into isometric coordinates to keep entity speeds unchanged, along with proper depth sorting and jump height calculations.

While data persistence is managed through a lightweight and human friendly JSON protocol. The user application directory (`user://savegame.save`) is a secure place to keep this protocol. The subsystem contains important player profile information like metadata, amount of gold, meta upgrades (max HP, movement speed, and base attack power) and snapshots of current dungeon activity so that the game could continue after sudden termination.

6.2 Project Game Development Setup

The Godot Engine version 4.6 was used to set up the development process, which included the GDScript Language Server Protocol (LSP) to ensure that the rigorous static type checking, real-time code linting, and error prevention. Besides, in order to allow simultaneous work on game features by four team members, the Git distributed version control system was used. The configuration of the repository was such that the `.gitignore` file was modified in order not to include cache folders, compiled editor binaries, or Godot import indices. In order to maintain the stability and avoid the bugs during iterations of game play features being added, the system uses an automated test execution framework,

which is organized within tests/ directory. The automated verification system performs a series of headless test runs on the game logic in order to verify that critical game loops are working, such as the conditions under which the gate unlock after the monster count reaches zero, the calculations of the damage that the explosive barrel can inflict, the time that the collapsed bridge goes through, and the transition of bosses between the waves.

6.3 Project Organization Overview

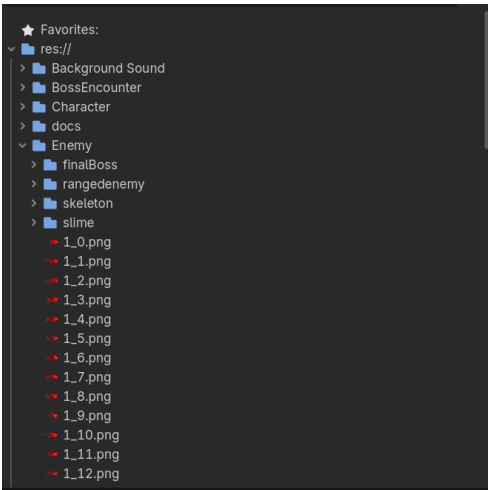


Figure 6.1: Project Directory and Asset Hierarchy in Godot 4.6 FileSystem Dock.

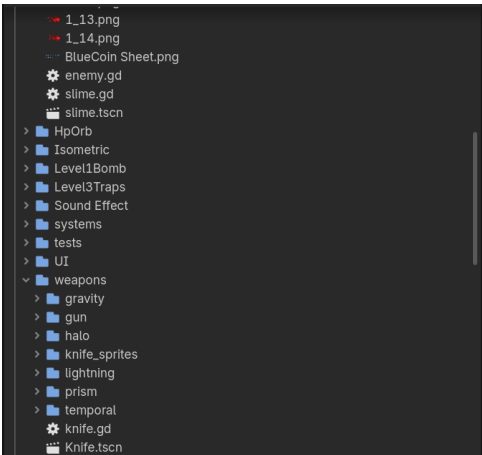
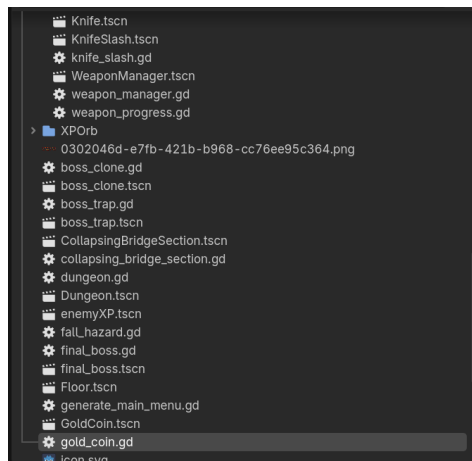
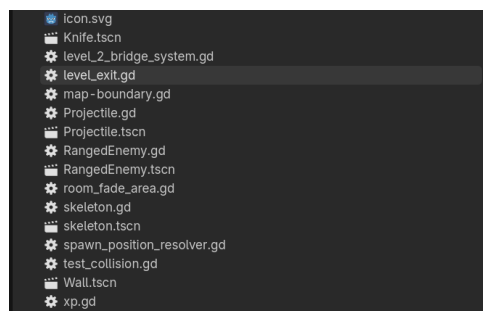


Figure 6.1: Project Directory and Asset Hierarchy in Godot 4.6 FileSystem Dock
(Continue)



*Figure 6.2: Project Directory and Asset Hierarchy in Godot 4.6 File System Dock
(Continue)*



*Figure 6.3: Project Directory and Asset Hierarchy in Godot 4.6 File System Dock
(Continue)*

The project directory is structured into modular domain-specific folders:

- Background Sound/ and Sound Effect/ : Store all the sound elements appropriately separated from background noises. Save elements in both .mp3 and .wav format for right distribution through the audio bus in the Godot program.
- Character/ and Enemy/ : Store in here the entire performance, animation and scripting elements of the actor. The folder of enemy is sorted into multiple specific subfolders such as slime, skeleton, ranged enemy, and final boss with their respective art and logical files.
- Weapons/ : Contains the 7 modular weapon subdirectories (Knife, Gun, Halo, Lightning, Gravity, Temporal, Prism)

- Level1Bomb/ , Level3Traps/ , BossEncounter/ : Contains scene hazards and room trial logic
- UI/ : Contains the Main Menu, Store, Minimap, HUD, and custom fonts (Milky Cream.otf and HARRYP_.TTF)
- systems/ : Houses global Autoload singletons (SaveSystem, AudioManager, TargetSelector)
- Dungeon.tscn & dungeon.gd: Act as the master gameplay scene and dungeon orchestration controller, managing room transitions, gate locks, enemy spawning, and overall level progression.

6.4 Game Flow / Scenes Overview

The runtime operation flow of Survive-till-Last is organized as game with state-based transitions among different interfaces, exploration and also combat scenes:

1. Initial Stage and Main Menu Flow:

When the game starts, its engine activates all global singletons, which including SaveSystem, WeaponProgress, AudioManager, and PauseMenu, before activating UI/MainMenu.tscn. At the main menu, players have an opportunity to manage their profiles like create, select, or delete player profiles, enter the persistent Meta-Progression Shop in order to invest collected Gold coin into permanent character upgrades, and start their dungeon expedition.

2. Dungeon Exploration and Level Progression:

When the run commences, it loads the master Dungeon.tscn location, which makes the player start from Level 1 (The Abandoned Vaults). As per the player clears through the monster waves and pops the explosive barrel, the dungeon manager keeps track of the real enemies alive. Clearing all slimes gives access to the exit and enables the player to see through Level 2 (The Chasm Bridges) on the vector HUD minimap. In Level 2, players make their way through falling bridge parts while battling the armored skeleton warriors.

Defeating Level 2 unlocks Level 3 (The Gauntlet of Trials), where players must evade synchronized spike rows, sweeping firefronts, and falling rock hazards while fighting ranged cultists.

3. Level 4 Boss Sanctum and Multi-Phase Encounter:

After completing Level 3 gives the players access to Level 4 (The Boss Sanctum). Once the players enter this level, the `BossEncounterController` is activated. The `BossEncounterController` runs a two-stage preparation sequence, where the players will fight against waves of slimes and spikes in the first wave, and waves of skeletons and fire in the second wave. Once the players finish both waves, the traps in the arena will be turned off, so they will be able to engage in the Final Boss. The final boss has three stages, Stage 1 with dodging tactics, Stage 2 with traps, and Stage 3 with clone summoning and crazy attacks.

4. Victory and Defeat Resolution:

Once the players successfully defeats the Final Boss, the victory music is played, followed by the display of the modal for `UI/GameWinUI.tscn`, which captures run statistics and redirects the player to the Main Menu. However, if the health of the player gets drained to zero while playing the dungeon run, the `player_died` signal sends the player to `UI/GameOverUI.tscn`, where he can either restart or head back to the Main Menu to upgrade character stats using the collected gold coins.

CHAPTER 77.0 Programming Conventions and Designs

The programming conventions and designs of Survive till Last are organizes to keep the game's scripts structured, consistent and easy to maintain. The project uses separate scripts for different gameplay systems. For example, player control, weapons, enemies, dungeon levels, environmental hazards, boss encounters and

user interface elements. This section describes the overall script setup, coding and naming conventions and the design patterns used throughout the project.

7.1 Overall Script Setup

The game scripts are organized according to their respective gameplay systems and responsibilities. The main scripts are separated into player control, weapons, enemies, dungeon and level systems, environmental hazards, boss encounters, progression systems, and user interface components. This allows each script to focus on a specific function while working together through Godot's scene and node system.

The player and gameplay scripts handle player movement, health, experience, weapons, enemy behavior, level progression, and environmental interactions. Separate scripts are also used for different weapons, projectiles, enemies, and level hazards to manage their individual behaviors. The project also contains supporting scripts for progression and other gameplay systems.

The user interface scripts manage the different interfaces used throughout the game, including the Main Menu, Pause Menu, Game Over, Victory screen, HUD, and dungeon mini map. Reusable UI components are used where appropriate to maintain a consistent interface structure.

Overall, the project follows a **modular script structure**, where related functionality is divided into separate scripts. This makes the code easier to organize, maintain, test, and modify during development.

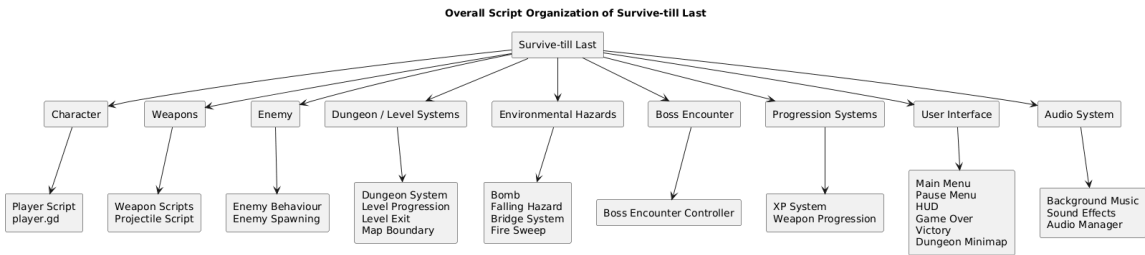


Figure 7.1: Overall Script Organization of Survive-till Last

7.2 Coding Conventions and Organization

The project uses GDScript coding conventions to keep the source code consistent and readable. Scripts are organized according to their gameplay responsibilities. The related scripts are placed in folders like Character, Enemy, Weapons, BossEncounter and UI. This separation allows each system to be developed and maintained independently.

The scripts make use of Godot features. For example, `@export` properties for configurable values, `@onready` variables for node references, signals for communication between components and `class_name` for defining reusable script classes. Typed variables and function declarations are also used in the project to improve code readability and reduce potential errors.

The following conventions are commonly used in the project:

Convention	Usage
snake_case	Used for script and variable names like <code>player.gd</code> and <code>xp.gd</code>
<code>@export</code>	Used for values that can be configured in the Godot Inspector
<code>@onready</code>	Used to initialize references to nodes
signal	Used for communication between different game components
class_name	Used to define reusable custom classes
Typed variables	Used to specify data types like <code>int</code> , <code>float</code> , <code>bool</code> and <code>Node2D</code>
Functions	Used to separate specific behaviours and operations
Comments or Section	Used to organize related blocks of code

Overall, the coding organization follows a modular and consistent approach. This allows the team members to work on different game systems while keeping the code easier to understand, maintain and debug.

7.3 Naming Conventions

The project follows consistent naming conventions to improve code readability, maintainability and organization. The naming style is mainly based on the conventions used in GDScript and Godot. Descriptive names are used for scripts, classes, variables, functions, signals and nodes. So, their purposes can be understood easily by other team members.

7.1 7.3.1 Script and File Naming

Script files use lowercase snake_case naming. The file name describes the main function or object controlled by the script.

Example include:

- player.gd
- dungeon.gd
- dungeon_minimap.gd
- fire_sweep.gd
- rock_strike.gd
- spike_row.gd
- guard_halo.gd
- temporal_echo.gd

Scene files generally use PascalCase when they represent major game objects or scenes. For example, Dungeon.tscn, DungeonMinimap.tscn and MainMenu.tscn.

7.2 7.3.2 Class Naming

Classes declared using class_name use PascalCase. The class name represents the object or system implemented by the script.

Examples from the project include:

Class	Purpose
FireSweep	Controls the fire sweep trap
RockStrike	Controls the failing rock hazard
SpikeRow	Controls the spike row trap
GuardHalo	Manages the player's defensive halo
TemporalEcho	Controls the temporal echo weapon

For example, the FireSweep script declares its class using `class_name FireSweep`, while the SpikeRow script uses `class_name SpikeRow`.

7.3 7.3.3 Variable and Function Naming

Variables and functions use **snake_case**, with descriptive names that indicate their purpose. This makes the code easier to understand when different systems interact with each other.

Example include:

- `warning_duration`
- `active_duration`
- `current_level`
- `current_xp`
- `player_died`
- `hp_changed`
- `damage_blocked`
- `_process()`
- `_physics_process()`

Exported variables also follow the same naming convention, such as `warning_duration` and `active_duration` in the SpikeRow script.

7.4 7.3.4 Signal Naming

Signals use **snake_case** and normally describe an event that has occurred. The project uses signals such as `sweep_finished`, `strike_finished`, `energy_changed`, `state_changed`, and `damage_blocked`.

7.5 7.3.5 Summary of Naming Conventions

Element	Naming Convention	Example
Script files	lowercase snake_case	dungeon_minimap.gd
Scene files	PascalCase	Dungeon.tscn
Classes	PascalCase	GuardHalo
Variables	snake_case	current_level
Functions	snake_case	_physics_process()
Signals	snake_case	damage_blocked
Exported variables	snake_case	warning_duration
Constants	Uppercase	MAX_HP

7.4 Overall Design Pattern Conventions

The project uses a modular design approach to separate major gameplay systems. For example, the player, enemies, weapons, dungeon, traps, UI and progression. The project also uses Godot Autoloads for global systems. For instance, the global systems are `SaveSystem`, `WeaponProgress`, `PauseMenu`, and `AudioManager`. In addition, signals are used for event-based communication between components. The enemy behaviours are organized using a state-based approach.

Design Pattern/Approach	Implementation
Modular Design	Separates gameplay systems into individual scripts and components

Autoload / Singleton	SaveSystem, WeaponProgress, PauseMenu, AudioManager
Signal Based Communication	Used for events such as sweep_finished and damage_blocked
State	Used to organize enemy behaviors and states

CHAPTER 88.0 Class Diagrams/UML Overview

8.1 System UML Class Diagram

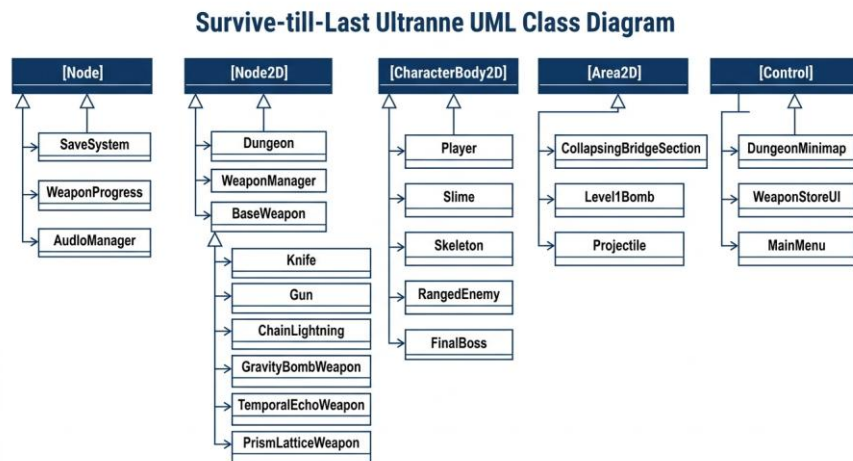


Figure 8.1: UML Class Diagram

8.2 Class List and Overview

- Player (CharacterBody2D): The primary controller for the player that allows for movements in 8 directions, parabolic jumping, and tracking for health/XP, weapons, and game end triggers.
- Dungeon (Node2D): The scene controller that oversees progress through the levels (Levels 1 through 4), enemy counts, gate crossing, and also win conditions.

- FinalBoss (CharacterBody2D): The boss that employs multiple phases, dash predicting, teleporting attacks, summoning clones, and conjuring laser blasts.
- WeaponManager (Node2D): The component linked to Player that handles weapon creation, switching using number keys between weapons 1 and 7, and weapon use.
- WeaponProgress (Node Autoload): The global development of all weapons that assesses conditions for unlocking weapons and also manages XP in the weapon shop.
- DungeonMiniMap (Control): The arcade map that translates coordinates into polygons to illustrate the movement of the Player and the enemies in the current level.
- SaveSystem (Node Autoload): The system that saves the progress of the user and also updates the parameters of the system using JSON technology.
- AudioManager (Node Autoload): Global audio manager for playback of BGM and sound effects without blocking.
- CollapsingBridgeSection (Area2D): Standalone hazard portion bearing time counters, collision cutoff, and fall detection.
- Level1Bomb (Area2D): Breakable bomb barrels that register impacts, countdown the time and cause area damage.

8.3 Methods Overview by Class

1. Player:

- `_physics_process(delta)`: Processes the user input through calculations of the amount of jump and also invokes the function `move_and_slide()`.
- `jump()`: Starts the parabolic movement and raises the sprite according to the height of jump.
- `take_damage(amount)`: Reduces the player's health (the damage is reduced by the Giard Halo), makes the player flash red, and sends a message about the hp change.

- `heal(amount)`: Heals some health but does not go beyond the max health limit of the player.
 - `gain_xp(amount)`: Receives experience points and levels up the character.
 - `equip_halo`: Equips the GuardHalo and turns it on.
2. **Dungeon:**
- `_ready()`: Initializes the level status and locks the gates of the level
 - `_on_enemy_killed`: Counts the amount of the living enemies and unlocks the gates in case no one else enemy left
 - `unlock_path_after_level(level_num)`: Disables the check of the gate and opens the next level
 - `_spawn_level_one_bombs()`: Creates the barrels in the level 1
 - `_on_boss_defeated()`: Plays the victory music and shows the Game Win Window
3. **FinalBoss:**
- `_physics_process(delta)`: Calculates the distance to target and executes relevant AI behavior for the phase in question.
 - `_transition_to_phase(new_phase)`: Adjusts the speed and also the behavioral parameters and activates attack pattern.
 - `_execute_melee_attack()`: Executes indicated frontal cleave.
 - `_execute_dash_attack()`: Anticipates the player movement and performs dash while creating a hitbox active during the attack.
 - `_execute_teleport_sweep()`: Reappears near to the player and creates energy waves.
 - `_spawn_boss_clones()`: Spawns illusory clones to mislead or support the player.
 - `_spawn_trap_pillars()`: Creates the pillars of arcane energy in various parts of the arena.

8.4 Variable Overview by Class

Variable	Class	Type	Function / Description
speed	Player	float (200.0)	Base player movement speed in pixels per second.
jump_height	Player	float (120.0)	Peak vertical visual displacement during a jump.
current_hp / max_hp	Player	int (100)	Current and maximum player health pools.
current_level	Dungeon	int (1–4)	Active dungeon level index.
profiles	SaveSystem	Dictionary	Serialized user profile records and upgrade stats.
WEAPON_DATA	WeaponProgress	Dictionary	Static weapon definitions (slot, cost, requirements).
health / max_health	FinalBoss	float (2000.0)	Boss health pool governing phase transitions

8.5 Feature Implementation Summary

- **Weapon Switching:** By measuring an input number of keystrokes (1-7) will help manage the Weapon Manager system's execution of the chosen weapon entity and prevent any others from operating.
- **Vector Mapping:** Some custom functions are created for smooth mapping of the Dungeon Minimap system's world to the screen through the invention of the Vector Minimap.
- **Damage Flow:** Different damage taking functions called `take_damage()` in order to make processes much more streamlined and efficient in various systems.

8.6 Game Object Design Setup

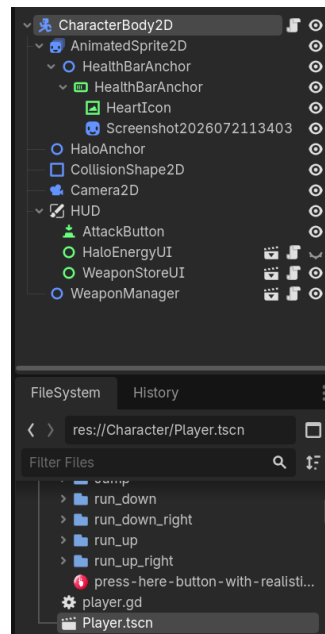


Figure 8.2: Player Scene Node Hierarchy

8.7 Player Controls Implementation Overview

Action	Key / Button	Functional Description
Arrow Keys	8-directional movement	Translates the player across the isometric floor using normalized 2D velocity vectors

Spacebar	Evasive Jump	Triggers visual parabolic elevation to leap over floor hazards (spikes, gaps) and avoid contact damage
1	Equip Knife	Selects the Knife. Automatically executes a close-range melee cleave against the nearest living enemy within 160px every 0.8s
2	Equip Gun	Select the Gun. Automatically fires high-velocity ballistic bullets with predictive trajectory leading at the closest enemy within 300px every 0.6s
4	Equip Chain Lightning	Select Chain Lightning. Automatically discharges electric arcs into the nearest enemy within 400px and branches to up to 4 neighboring targets every 0.6s
5	Cast Gravity Bomb	Instantly throws a gravity grenade that pulls nearby enemies into a vortex singularity before detonating in a high-damage radial blast

6	Cast Temporal Echo	Instantly projects a ghost decoy replaying the player's previous 5 seconds of movement, drawing enemy aggro and pulsing area damage
7	Cast Astral Prism Snare	Instantly launch an autonomous hunting prism that seeks the nearest enemy, trapping it inside a damaging laser containment lattice
Passive	Guard Halo Shield	Always active in the background once unblocked and automatically absorbs 30% of incoming damage through a regenerating energy pool
Escape / Menu Button	Pause Menu	Toggle the pause menu overlay and audio settings modal

CHAPTER 99.0 Audio Implementation

Overview

9.1 Audio File Setup & Bus Hierarchy

Audio playback is managed locally by the AudioManager.gd singleton (PROCESS_MODE_ALWAYS) using a 3-tier bus structure:

1. Master Bus: The root audio channel responsible for the total application output and master volume attenuation.
2. Music Bus: Dedicated channel routing all looping environmental background music. Supports independent volume scaling and automatic track suppression during victory or defeat sequences.
3. SFX Bus: Manages all the in-game sound effects using a 16-voice AudioStreamPlayer pool in order to ensure that concurrent, non-overlapping playback during the intense combat.

Players can adjust the volume smoothly using the volume sliders in the Pause Menu by using the logarithmic conversion: `linear_to_db(value)`.

9.2 Sound Effects Overview

Audio Asset	File Format and Spec	File Size	Audio Bus	In-Game Trigger /Purpose
Coin Collection	WAV (44.1 kHz, 16-bit)	176 KB	SFX	Plays chime when collecting Gold Coins.
Level Clear	WAV (44.1 kHz, 16-bit)	515 KB	SFX	Plays room clear fanfare
Hit Impact	WAV (44.1 kHz, 16-bit)	496 KB	SFX	Plays hit sound on player/enemy damage when using knife weapon.

Player Defeat	WAV (44.1 kHz, 16-bit)	517 KB	SFX	Stops BGM and plays game over sting on death.
Victory	WAV (44.1 kHz, 16-bit)	1.40 MB	SFX	Stops BGM and plays victory fanfare upon beating the boss.

9.3 Background Track Overview

Audio Track	File Format and Spec	File Size	Audio Bus	In-Game Usage
Obsidian Halls	MP3 (44.1 kHz, Stereo)	7.53 MB	Music	Looping ambient background music across Levels 1-4

The background track using the MP3 format in order to reduce the file size while also maintaining atmospheric audio quality. It loops continuously throughout the dungeon run until the player dies or defeats the final boss.

CHAPTER 1010.0 Game AI Overview

10.1 Game AI Overview & State Machine Script Architecture

All enemy AI scripts use two main systems, which are finite state machines and combat target selectors. Each enemy uses a state to control its behavior. The current state is checked for every frame inside `_physics_process()` using `match current_state` block. All enemies use the shared `combat_target_selector.gd` to

determine their target. The target is checked regularly instead of keeping a constant reference to the player. Before considering an object's valid combat target, the system also verifies that the enemy is alive and belongs to the correct enemy group.

10.2 Enemy Type 1: Slime (slime.gd)

The slimes are the basic melee group enemy that chases the player together while using different movement patterns. When the player enters its detection range, the enemy changes from wandering to chasing. When it gets close enough, it performs a melee attack. The AI also prevents the slimes from overlapping and helps them move around the obstacles.

Main States:

1. WANDER - moves to random positions around its starting location and
2. CHASE - trigger when the player enters the VisionArea. The slime will move toward the player
3. ATTACK – trigger when the player within attack_range, the slime stops and performs a melee attack
4. DEATH – the slime stops moving and performs the death animation.

The slime uses a State enum to control its behavior. A zigzag movement is added during chasing to make different slimes take slightly different paths. The slime uses `_get_obstacle_aware_direction()` function to detect the wall. If the path is blocked, the AI checks another possible direction around the obstacle and chooses a clearer path. When the enemy dies, it will drop both XPOrb (XP.tscn) and coin (GoldCoin.tscn).

10.3 Enemy Type 2: Skeleton (skeleton.gd)

The skeleton is a stronger melee enemy that introduces the combo attack system. It also responds to nearby allies, to enable multiple skeletons to attack player together.

Main States:

1. IDLE – waits before starting to move
2. WANDER – moves around its starting area
3. CHASE – follows the player
4. ATTACK – performs a normal attack or a second combo attack
5. RETREAT – temporarily moves away from the player
6. ALERT – responds to alert from another enemy

The skeleton uses ComboStage system with three stages, which are NONE, ATTACK_1 and ATTACK_2. After the first attack, a combo_timer starts, If the player remains within the required range before the timer expires, the skeleton can perform ATTACK_2. Attack 2 has a larger range and higher damage than the first attack. The attack damage is applied also when the attack animation reaches the specific damage frame.

10.4 Enemy Type 3: Ranged Enemy (RangedEnemy.gd)

The ranged enemy is designed to attack the player from a distance. It will maintain a suitable shooting distance instead of moving directly toward the player all the time. When the player gets too close, it retreats to maintain the distance with the player while continuing to attack.

Main States:

1. IDLE – waits before return to normal movement
2. WANDER – move around its home area
3. CHASE – moves closer to the player when the player is too far away
4. SHOOT – stops and attack player with fire balls
5. RETREAT - moves away when the player gets too close
6. STUN - temporarily stops after getting hurts
7. ALERT- responds to nearby enemy alerts
8. DEATH – performs death animation
9. RETURN_HOME – returns to its starting area when it moves too far away

The ranged enemy uses 3 main distance values, which are `min_range`, `ideal_range`, and `attack_range`. If the player is too far away, the enemy enters CHASE. It enters SHOOT state, when the player reaches a suitable distance to shoot. If the players get inside `min_range`, the enemy enters RETREAT. The enemy uses `_fire_at_player()` function to attack players with fire balls. The firing speed change depends on the distance between the enemy and the player. The retreat system also checks whether the enemy is stuck at the corner and cannot move too far away. It will change behavior instead of repeatedly trying to move through obstacles.

The projectile system (`projectile.gd`) is used by ranged attacks to create moving projectiles that can damage the player. The projectile moves in its assigned direction using its speed value. Its rotation is also updated so that it always faces the direction it is towards. When a projectile spawns, it plays a flight animation and connects the `body_entered` signal to detect collisions. Once hit on the player, it calls `take_damage()` and creates hit effect. If it hits a wall, such as `TileMapLayer` or `StaticBody2D`, it also creates the hit effect and is removed. The projectile also has limited lifetime so it is automatically removed if it does not hit anything before the lifetime ends.

10.5 Enemy Type 4: Final Boss (`final_Boss.gd`)

The final boss is the most advanced AI system in the game. It uses 3 phases with different behaviors and attack methods.

The three phases are:

- Phase 1 : 100% -60% health
- Phase 2: 60% - 30% health
- Phase 3: Below 30% health

Main States:

1. IDLE – wait before performing any action
2. CHASE – moves toward player

3. CIRCLE – moves around the player
4. ATTACK – performs melee attacks
5. DASH – quickly move towards player and can deal damage
6. TELEPORT – teleports near player
7. COUNTER – responds to player's attack
8. DEATH – when its health reaches zero

Phase 1 : Melee and Chase

In phase 1, the boss performs basic melee attacks. The `predictive_chase()` function predicts the player's possible next position by using the player's velocity and `prediction_strength`. This allows the boss to move toward the predicted player position instead of simply chasing the player's current position. The boss can also perform a short dash when the player is within a specific distance range. After the dash, it performs a melee attack.

Phase 2: Teleport and Movement

In phase 2, the boss movement increases and more different attack methods. If the player is very far away from the boss, it can perform a teleport. The boss disappears, moves to a position close to the player, and quickly performs an attack. After the teleport attack, the boss also drops a trap near the player. The teleport also has a cooldown time so the boss cannot use teleport continuously. The boss can move around the player instead of always moving directly toward them. This makes their movement less predictable and more challenging for players. The boss may also use a dash attack in this phase.

The trap (`boss_trap.gd`) is a temporary plant attack used to surprise the player. First, the trap plays its seed animation. It grows and then enters an idle state after the `seed_delay`. Once the trap is ready to attack, it checks if the target is within `attack_range` and performs attacks. After attack animation finishes, the trap

disappears. The trap can only attack once due to the `has_attacked` preventing it from performing multiple attacks.

Phase 3: Enraged final phase

When the boss reaches below 30% health, it enters phase 3. This is the most difficult phase because it combines several previous abilities and adds additional attacks. Once entering phase 3, the boss summons 2 boss clones.

The clones (`boss_clone.gd`) are a temporary melee enemy to pressure on player. It does not use traditional State enum like the previous enemy use, it uses Boolean variables such as `appearing`, `attacking` and `dying`. It has a limited lifetime and disappears and is removed from the scene when the lifetime ends. These boss clones can attack players, but they cannot be damaged by the player.

The boss also had a stronger melee attack. Its combo system allows multiple attacks, the damage also increased. The boss also features a counterattack mechanic. When the player attacks, the boss has a chance to block the attack and counterattack. The chance increases with each phase: 15% in Phase 1, 20% in Phase 2, and 25% in Phase 3.

10.6 Weapon Type

10.1 10.6.1 Weapon 1: Knife

- The Knife is the player's starter weapon and is assigned to number key 1.
- It is an automatic close-range weapon.
- It searches for the nearest living enemy within 160 pixels.
- It deals 30 base damage and has a 0.8-second cooldown.
- Each attack creates a short-lived slash hitbox in front of the player.
- The same slash cannot damage the same enemy more than once.

Weapon Configuration

```

6  @export var damage: int = 30
7  @export var attack_cooldown: float = 0.8
8  @export var attack_range: float = 160.0
9  @export var knife_slash_scene: PackedScene
10 |
11  var cooldown_left: float = 0.0
12  @onready var player = get_tree().get_first_node_in_group("player")

```

How it works:

The Knife uses several variables to control its behaviour. The damage variable determines how much damage is dealt to an enemy, while attack_cooldown limits the Knife to one attack every 0.8 seconds. The attack_range variable defines the maximum distance within which the weapon can detect an enemy. The knife_slash_scene stores the scene containing the slash animation and collision area used during an attack. Meanwhile, cooldown_left tracks how much time remains before the Knife can attack again. Finally, the weapon locates the player node through the "player" group so that it can follow the player's position and access the player's attack information.

```

17  func _physics_process(delta: float) -> void:|
18  >| if player and "current_hp" in player and player.current_hp <= 0:
19  >| >| return
20
21  >| cooldown_left -= delta
22  >| if cooldown_left > 0.0:
23  >| >| return
24
25  >| var target = _find_nearest_enemy_in_range()
26  >| if target != null:
27  >| >| do_attack(target)

```

How it works:

This function runs continuously during each physics update to control the Knife's automatic attack. It first checks whether the player has died and stops the weapon from attacking if the player's health is zero. The function then

decreases `cooldown_left` using `delta`, which represents the time passed since the previous physics frame. When the cooldown reaches zero, the weapon searches through all nodes in the "enemy" group while ignoring invalid or defeated enemies. It calculates the distance between the Knife and each valid enemy, then selects the nearest enemy within the 160-pixel attack range. If a suitable target is found, the `do_attack()` function is called to begin the Knife attack.

```
41  ▾ func do_attack(target: Node2D = null) -> void:|
42  ▾ >| if cooldown_left > 0.0:
43  >| >| return
44  >| cooldown_left = attack_cooldown
45
46  >| var dir: Vector2
47  ▾ >| if target != null:
48  >| >| dir = (target.global_position - global_position).normalized()
49  ▾ >| else:
50  >| >| dir = player.last_direction.normalized()
51
52  ▾ >| if knife_slash_scene:
53  >| >| _spawn_projectile(dir)
54  >|
55  ▾ >| if player.has_method("play_attack_animation"):
56  >| >| player.play_attack_animation()
57
58  ▾ func _spawn_projectile(dir: Vector2) -> void:
59  >| var slash = knife_slash_scene.instantiate()
60  >| get_tree().current_scene.add_child(slash)
61  >| slash.global_position = global_position
62  >| slash.damage = damage
63  >| slash.direction = dir
```

How it works:

When an attack begins, the cooldown is reset to 0.8 seconds to prevent the Knife from attacking again immediately. The weapon calculates a direction vector from the Knife towards the selected target; if no target is available, it uses the player's last movement direction. The `instantiate()` function then creates a new Knife slash from `KnifeSlash.tscn` and adds it to the current level

so it can operate independently. The slash begins at the player's position and receives the Knife's damage value of 30. At the same time, the player's attack animation is played to provide visual feedback for the attack.

```
16  ▾ func _ready() -> void:|
17  >| hit_area.body_entered.connect(_on_body_entered)
18  >| hit_area.area_entered.connect(_on_area_entered)
19  >|
20  >| # FORCE VISUAL FIXES (bypasses Godot editor wiping out .tscn changes)
21  >| sprite.scale = Vector2(0.4, 0.4)
22  >| var mat = CanvasItemMaterial.new()
23  >| mat.blend_mode = CanvasItemMaterial.BLEND_MODE_ADD
24  >| sprite.material = mat
25  >|
26  >| # Force radius fix
27  ▾ >| if hit_area.has_node("CollisionShape2D"):
28  >| >| var shape_node = hit_area.get_node("CollisionShape2D")
29  ▾ >| >| if shape_node.shape == null:
30  >| >| >| shape_node.shape = CircleShape2D.new()
31  >| >| shape_node.shape.radius = 80.0
```

```

33  >|  # Offset the stationary slash in front of the player
34  >|  global_position += direction * 100.0
35  >|
36  >|  # Flip sprite if attacking left
37  >|  if direction.x < 0:
38  >|  >|  sprite.flip_h = true
39  >|  else:
40  >|  >|  sprite.flip_h = false
41  >|  >|
42  >|  sprite.play("attack")
43  >|  await get_tree().create_timer(lifetime).timeout
44  >|  queue_free()

```

Collision signals detect when enemy bodies or attack areas enter the Knife slash. The slash uses a circular hitbox with a radius of 80 pixels and is positioned 100 pixels in front of the player. When the player attacks towards the left, the sprite is flipped horizontally so that the visual effect faces the correct direction. The "attack" animation then displays the four Knife slash frames. The slash remains active for only 0.2 seconds before `queue_free()` removes it from the scene, preventing expired attack objects from remaining in memory.

```

49  var hit_enemies: Array = []
50
51  >| func _on_body_entered(body: Node2D) -> void:
52  >|  print("[Knife] body_entered: ", body.name, ", groups: ", body.get_groups())
53  >|  if body in hit_enemies: return
54  >|  if COMBAT_TARGET_SELECTOR.is_living_enemy(body):
55  >|  >|  print("[Knife] HIT ENEMY BODY!")
56  >|  >|  body.take_damage(WEAPON_DAMAGE.calculate(damage, self))
57  >|  >|  hit_enemies.append(body)
58  >|  >|  if get_node_or_null("/root/AudioManager"): get_node("/root/AudioManager").

```

The `hit_enemies` array records every enemy that has already been struck by the current slash, preventing repeated collision events from damaging the same enemy multiple times. When a collision occurs, the target selector first checks that the detected body is a living enemy. The `take_damage()` function then applies the calculated weapon damage, after which the enemy is added

to the `hit_enemies` array. A blood-impact sound is also played through the `AudioManager` to provide audio feedback. During the final-boss stage, the shared damage calculator increases the Knife's damage by 50%, raising its normal damage from 30 to 45.

10.2 10.6.2 Weapon 2: Gun

- The Gun is a ranged automatic weapon and assigned to number key 2.
- It becomes available in Level 2 after the player completes Level 1.
- It automatically targets the nearest living enemy within 300 pixels.
- It fires one bullet every 0.6 seconds.
- Each bullet deals with 30 base damage.
- The Gun predicts the enemy's movement to improve its accuracy.

```

28  ▾ func _physics_process(delta: float) -> void:|
29      >|    cooldown_left -= delta
30  ▾ >|    # Input is visible to physics before WeaponManager completes its normal
31      >|    # process-frame switch. Suppress a final Gun shot on that transition frame.
32  ▾ >|    if _is_switching_away_from_gun():
33      >|        >|    return
34  ▾ >|    if cooldown_left > 0.0:
35      >|        >|    return
36      >|    var target = _find_nearest_enemy_in_range()
37  ▾ >|    if target != null:
38      >|        >|    do_attack(target)

```

```

41  ▾ func _is_switching_away_from_gun() -> bool:
42  ▾ >|    for action in NON_GUN_WEAPON_ACTIONS:
43  ▾ >|        >|    if Input.is_action_just_pressed(action):
44      >|        >|        >|    return true
45      >|    return false

```

```

47  ▾ func _find_nearest_enemy_in_range() -> Node2D:
48      >|    var nearest: Node2D = null
49      >|    var nearest_dist := attack_range
50  ▾ >|    for enemy in get_tree().get_nodes_in_group("enemy"):
51  ▾ >|        >|    if not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
52      >|        >|        >|    continue
53      >|        >|    var dist = global_position.distance_to(enemy.global_position)
54  ▾ >|        >|    if dist <= nearest_dist:
55      >|        >|        >|    nearest_dist = dist
56      >|        >|        >|    nearest = enemy
57      >|    return nearest

```

Automatic target detection and firing process of the Gun.

The Gun updates its cooldown during every physics frame. It does not attack while its cooldown is active or while the player is switching to another weapon. When it is ready, it examines the nodes in the "enemy" group and ignores any enemy that is no longer alive. The distance between the Gun and each valid enemy is calculated, allowing the nearest enemy inside the 300-pixel attack range to be selected. The selected enemy is then passed to the `do_attack()` function.


```

59  ▾ func do_attack(target: Node2D = null) -> void:
60  ▾  >|  if cooldown_left > 0.0:
61      >|  >|  return
62      >|  cooldown_left = attack_cooldown
63
64      >|  var dir: Vector2
65  ▾  >|  if is_instance_valid(target):
66      >|  >|  dir = (_predict_target_position(target) - global_position).normalized()
67  ▾  >|  elif player != null and "last_direction" in player:
68      >|  >|  dir = player.last_direction.normalized()
69  ▾  >|  else:
70      >|  >|  dir = Vector2.RIGHT
71
72      >|  _spawn_bullet(dir, target)
73  ▾  >|  if player != null and player.has_method("play_shoot_animation"):
74      >|  >|  player.play_shoot_animation()

```

```

77  ▾ func _predict_target_position(target: Node2D) -> Vector2:
78      >|  var aim_position := _get_target_aim_position(target)
79  ▾  >|  if target is CharacterBody2D:
80  ▾  >|  >|  var travel_time := minf(
81      >|  >|  >|  global_position.distance_to(aim_position) / maxf(bullet_speed, 1.0),
82      >|  >|  >|  maximum_prediction_time
83      >|  >|  )
84      >|  >|  aim_position += target.velocity * travel_time * prediction_strength
85      >|  return aim_position

```

When an attack begins, the cooldown is reset to 0.6 seconds. Instead of firing directly at the enemy's current position, the Gun estimates where the enemy will be when the bullet arrives. It calculates the expected travel time using the distance and bullet speed, then adds part of the enemy's velocity to the aiming position. The prediction is limited to 0.65 seconds and uses a strength of 0.8, which prevents excessive prediction. The final direction is normalized before the bullet is created, and the player's shooting animation is played.

```

36  ▾ func _physics_process(delta: float) -> void:
37  ▾  » # Prediction supplies the main trajectory. Homing only makes a small early
38  » # correction; it switches off near the target or after an overshoot so a
39  » # bullet can never loop around and fly backwards.
40  » homing_time_left = maxf(0.0, homing_time_left - delta)
41  ▾  » if homing_time_left > 0.0 and is_instance_valid(target) \
42  »   » and COMBAT_TARGET_SELECTOR.is_living_enemy(target):
43  »   » var aim_position := _get_target_aim_position(target)
44  »   » var to_target := aim_position - global_position
45  »   » var desired_direction := to_target.normalized()
46  »   » var target_is_ahead := direction.dot(desired_direction) > 0.2
47  ▾  »   » if to_target.length() <= homing_stop_distance or not target_is_ahead:
48  »   »   » target = null
49  »   »   » homing_time_left = 0.0
50  ▾  »   » elif desired_direction != Vector2.ZERO:
51  ▾  »   »   » var turn_amount := clampf(
52  »   »   »   » direction.angle_to(desired_direction),
53  »   »   »   » -homing_turn_speed * delta,
54  »   »   »   » homing_turn_speed * delta
55  »   »   » )
56  »   »   » direction = direction.rotated(turn_amount).normalized()
57  »   »   » rotation = direction.angle()
58  »   global_position += direction * speed * delta

```

Bullet movement and temporary homing correction.

How it works:

After being fired, the bullet receives a short 0.32-second homing period. During this period, it gradually rotates towards the selected enemy instead of changing direction immediately. Homing stops when the bullet is within 52 pixels of the target or after it has passed the enemy. This restriction prevents the bullet from turning around or circling the target. Finally, the bullet's position is updated using its direction, speed and frame time.

```

67  var hit_enemies: Array = []
68
69  ▾ func _on_body_entered(body: Node2D) -> void:
70  »   if body in hit_enemies: return
71  ▾  »   if COMBAT_TARGET_SELECTOR.is_living_enemy(body):
72  »   »   body.take_damage(WEAPON_DAMAGE.calculate(damage, self))
73  »   »   hit_enemies.append(body)

```

Gun bullet collision detection and damage application.

How it works:

When the bullet enters an enemy's collision body, the code first checks whether that enemy has already been hit by the same bullet. A valid living enemy receives the calculated weapon damage and is then stored in `hit_enemies`, preventing repeated damage from multiple collision signals. The bullet is not immediately destroyed after a hit, so it can continue travelling and potentially damage another enemy. Its total lifetime is 1.5 seconds, after which it is automatically removed. During the final-boss stage, the shared damage calculator increases the Gun's damage from 30 to 45.

10.3 10.6.3 Weapon 3: Guard Halo

- The Guard Halo is a passive defensive weapon that unlocks when the player reaches Level 3 and does not require manual selection.
- Once unlocked, six energy orbs automatically rotate around the player.
- The Halo absorbs part of the incoming damage, while its rotating orbs deal 15 contact damage to nearby enemies.

```
138 func get_current_absorption_ratio() -> float:
139     >| var energy_limit := maxf(max_energy, 0.0)
140     >| if energy_limit <= 0.0:
141         >| return 0.0
142
143     >| var energy_ratio := clampf(current_energy / energy_limit, 0.0, 1.0)
144     >| if energy_ratio >= high_energy_threshold:
145         >| return clampf(high_absorption_ratio, 0.0, 1.0)
146     >| if energy_ratio > low_energy_threshold:
147         >| return clampf(medium_absorption_ratio, 0.0, 1.0)
148     >| return clampf(low_absorption_ratio, 0.0, 1.0)
```

Selection of the Guard Halo's damage-absorption level based on its remaining energy.

How it works:

The Halo's protection becomes weaker as its energy decreases. When at least 70% of its energy remains, it absorbs 70% of incoming damage. Between 30% and 70% energy, it absorbs 50%, while at 30% energy or lower it absorbs 30%. This system makes the Halo strongest when fully charged while still providing limited protection when nearly depleted.

```
153  func absorb_damage(incoming_damage: int) -> int:
154  >|  if incoming_damage <= 0:
155  >|  >|  return 0
156
157  >|  if state != HaloState.ACTIVE:
158  >|  >|  return incoming_damage
159
160  >|  # Calculate the portion this Halo is allowed to block. Damage is integer-
161  >|  # based, so round to the closest whole damage point.
162  >|  var desired_blocked_damage := roundi(
163  >|  >|  float(incoming_damage) * get_current_absorption_ratio()
164  >|  >|  )
165  >|  if desired_blocked_damage <= 0:
166  >|  >|  return incoming_damage
167
168  >|  # Prevent an invalid Inspector value from causing division by zero.
169  >|  var cost_per_damage := maxf(energy_cost_multiplier, 0.001)
170  >|  var energy_capacity := floori(current_energy / cost_per_damage)
171  >|  var blocked_damage := mini(
172  >|  >|  desired_blocked_damage,
173  >|  >|  energy_capacity
174  >|  >|  )
```

```

176  ▾ ▸ | # If the remaining energy cannot fund one complete damage point, consume it
177  ▸ | # and break instead of leaving the Halo permanently active with unusable energ
178  ▾ ▸ | if blocked_damage <= 0:
179  ▸ |     ▸ | break_halo()
180  ▸ |     ▸ | return incoming_damage
181
182  ▸ | var consumed_energy := float(blocked_damage) * cost_per_damage
183  ▸ | current_energy = maxf(0.0, current_energy - consumed_energy)
184  ▸ | var remaining_damage := maxi(0, incoming_damage - blocked_damage)
185
186  ▸ | energy_changed.emit(current_energy, max_energy)
187  ▾ ▸ | if blocked_damage > 0:
188  ▸ |     ▸ | damage_blocked.emit(blocked_damage)
189  ▸ |     ▸ | _play_block_effect()
190
191  ▾ ▸ | if current_energy <= 0.0:
192  ▸ |     ▸ | break_halo()
193
194  ▸ | return remaining_damage

```

Calculation of blocked damage and Guard Halo energy consumption.

How it works:

When the player is attacked, `absorb_damage()` calculates how much damage the active Halo is allowed to block. The amount is limited by both the current absorption ratio and the available shield energy. By default, blocking one point of damage consumes one energy point. For example, a full-energy Halo receiving 10 damage blocks 7 points, consumes 7 energy and returns only 3 damage to the player. When the remaining energy reaches zero, `break_halo()` changes the Halo to its broken state.

```

103     >| >| energy_changed.emit(current_energy, max_energy)
104     >| change_state(HaloState.BROKEN)
105     >| func _on_broken_timer_timeout() -> void:
106     >| if state == HaloState.BROKEN:
107     >| >| change_state(HaloState.RECHARGING)
108
109     >| func _process(delta: float) -> void:
110     >| if state != HaloState.RECHARGING:
111     >| >| return
112
113     >| var energy_limit := maxf(max_energy, 0.0)
114     >| var energy_gained := maxf(recharge_rate, 0.0) * delta
115     >| var next_energy := minf(current_energy + energy_gained, energy_limit)
116
117     >| if next_energy >= energy_limit:
118     >| >| restore_halo()
119     >| >| return
120
121     >| if not is_equal_approx(next_energy, current_energy):
122     >| >| current_energy = next_energy
123     >| >| energy_changed.emit(current_energy, max_energy)
124
125     >| func restore_halo() -> void:
126     >| if state != HaloState.RECHARGING:
127     >| >| return
128
129     >| current_energy = maxf(max_energy, 0.0)
130     >| energy_changed.emit(current_energy, max_energy)
131     >| change_state(HaloState.ACTIVE)

```

Guard Halo recharge process after the shield is broken.

When its energy is exhausted, the Halo enters the BROKEN state and its orbs disappear. After a three-second delay, it changes to the RECHARGING state. Energy then regenerates at 20 points per second until the maximum of 100 is reached. During both the broken and recharging states, incoming damage passes directly to the player and the orbs cannot damage enemies. Once fully recharged, the Halo returns to the active state and restores its protection and orbiting orbs.

```

34  ▾ func _process(delta: float) -> void:
35  ▾ >|  if not damage_enabled:
36      >|  >|  return
37
38      >|  # tick down each enemy's cooldown
39  ▾ >|  for enemy in _recent_hits.keys():
40      >|  >|  _recent_hits[enemy] -= delta
41  ▾ >|  >|  if _recent_hits[enemy] <= 0.0:
42      >|  >|  >|  _recent_hits.erase(enemy)
43
44      >|  # check everything currently overlapping this orb
45  ▾ >|  for body in get_overlapping_bodies():
46      >|  >|  _try_damage(body)
47
48  ▾ func _try_damage(body: Node2D) -> void:
49  ▾ >|  if not damage_enabled:
50      >|  >|  return
51  ▾ >|  if not COMBAT_TARGET_SELECTOR.is_living_enemy(body):
52      >|  >|  return
53  ▾ >|  if _recent_hits.has(body):
54      >|  >|  return
55      >|  _play_heat_damage_effect(body)
56      >|  body.take_damage(WEAPON_DAMAGE.calculate(damage, self))
57      >|  _recent_hits[body] = hit_cooldown

```

Enemy detection and contact-damage behaviour of each Guard Halo orb.

How it works:

Each orb continuously checks for overlapping enemies. When a living enemy touches an orb, it receives 15 damage and a visual heat effect. The enemy is then recorded in `_recent_hits` for 0.5 seconds, preventing the same orb from applying damage every frame. After the cooldown expires, that orb can damage the enemy again. Because each of the six orbs maintains its own hit record, different orbs can damage the same enemy as they rotate around the player.

10.4 10.6.4 Weapon 4 : Chain Lightning

- Chain Lightning is an automatic multi-target weapon assigned to number key 4, and it unlocks when the player reaches Level 4.
- It attacks every 1.2 seconds by selecting the nearest enemy within 400 pixels as the primary target.
- The lightning can branch to four additional enemies within 300 pixels of the primary target, dealing 15 damage to each affected enemy.

```
34  ▾ func _physics_process(delta: float) -> void:
35    >|  cooldown_left -= delta
36  ▾ >|  if cooldown_left > 0.0:
37    >|    >|  return
38    >|  var first = _find_nearest_enemy(global_position, attack_range, [])
39  ▾ >|  if first != null:
40    >|    >|  do_attack(first)
```

Automatic primary-target detection for the Chain Lightning weapon.

The weapon continuously reduces its remaining cooldown. Once the cooldown reaches zero, `_find_nearest_enemy()` searches for the closest living enemy within the 400-pixel attack range. If a valid target is found, it is passed to `do_attack()` as the primary target. Therefore, the weapon operates automatically while selected and does not require a separate attack button.

```
44  ▾ func do_attack(first_target: Node2D) -> void:
45  ▾ >|  if not COMBAT_TARGET_SELECTOR.is_living_enemy(first_target):
46    >|    >|  return
47    >|  cooldown_left = attack_cooldown
48    >|  var hit_targets: Array[Node2D] = [first_target]
49    >|  var secondary_limit := maxi(max_jumps - 1, 0)
50  ▾ >|  hit_targets.append_array(_find_enemies_around(
51    >|    >|  first_target.global_position,
52    >|    >|  jump_range,
53    >|    >|  hit_targets,
54    >|    >|  secondary_limit
55    >|  ))
```



```

59  >|  # Capture every endpoint before damage in case an enemy dies immediately.
60  ▾>|  var bolt_segments: Array[PackedVector2Array] = [PackedVector2Array([
61  >|  >|  global_position,
62  >|  >|  first_target.global_position,
63  >|  ])]
64  ▾>|  for index in range(1, hit_targets.size()):
65  ▾>|  >|  bolt_segments.append(PackedVector2Array([
66  >|  >|  >|  first_target.global_position,
67  >|  >|  >|  hit_targets[index].global_position,
68  >|  >|  ]))
69
70  ▾>|  for target in hit_targets:
71  ▾>|  >|  if COMBAT_TARGET_SELECTOR.is_living_enemy(target):
72  >|  >|  >|  target.take_damage(WEAPON_DAMAGE.calculate(damage, self))
73
74  >|  _draw_bolts(bolt_segments)

```

Construction of the Chain Lightning target list, damage application and bolt segments.

The primary enemy is placed into `hit_targets`, and the weapon calculates how many additional enemies may be selected. Since `max_jumps` is 5, the attack can damage one primary enemy and up to four secondary enemies. The first bolt segment connects the player to the primary target, while every additional segment connects the primary target to a secondary enemy. Damage is applied immediately to all valid targets before `_draw_bolts()` produces the electrical effect. This means the lightning lines visualize the attack but are not responsible for collision detection.

```

79  ▾ func _find_enemies_around(
80    ▸ origin: Vector2,
81    ▸ max_dist: float,
82    ▸ exclude: Array[Node2D],
83    ▸ limit: int
84  ▾ ) -> Array[Node2D]:
85    ▸ var candidates: Array[Node2D] = []
86  ▾ ▸ if limit <= 0:
87    ▸ ▸ return candidates
88
89  ▾ ▸ for node in get_tree().get_nodes_in_group("enemy"):
90    ▸ ▸ var enemy := node as Node2D
91  ▾ ▸ ▸ if enemy == null or enemy in exclude:
92    ▸ ▸ ▸ continue
93  ▾ ▸ ▸ if not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
94    ▸ ▸ ▸ continue
95  ▾ ▸ ▸ if origin.distance_to(enemy.global_position) <= max_dist:
96    ▸ ▸ ▸ candidates.append(enemy)
97
98  ▾ ▸ candidates.sort_custom(func(a: Node2D, b: Node2D) -> bool:
99    ▸ ▸ return origin.distance_squared_to(a.global_position) < origin.distance_squ
100  ▸ )
101  ▾ ▸ if candidates.size() > limit:
102    ▸ ▸ candidates.resize(limit)
103  ▸ return candidates

```

Selection of the nearest secondary targets surrounding the primary enemy.

The function searches for enemies within 300 pixels of the primary target. It excludes the primary enemy and any target that has already been selected, ensuring that an enemy cannot be damaged twice during the same attack. All valid candidates are sorted according to their distance from the primary target. The list is then shortened to the permitted number of secondary enemies, producing compact and predictable lightning branches.

Although the weapon is called Chain Lightning, the implementation uses a hub structure rather than a serial chain. The player's bolt connects to the primary enemy, and the remaining bolts spread outward from that primary enemy to the nearby secondary targets.

```

181  ▾ func _jagged_points(start: Vector2, end: Vector2) -> Array:
182    >|  var result: Array = []
183    >|  var segments := 8          # more = finer jaggedness
184    >|  var jaggedness := 28.0     # bigger = wilder bolt
185    >|  var direction = (end - start)
186    >|  var normal = direction.orthogonal().normalized()
187    >|  result.append(start)
188  ▾ >|  for s in range(1, segments):
189    >|    >|  var t = float(s) / segments
190    >|    >|  var point_on_line = start.lerp(end, t)
191    >|    >|  var offset = normal * randf_range(-jaggedness, jaggedness)
192    >|    >|  result.append(point_on_line + offset)
193    >|  result.append(end)
194    >|  return result

```

Generation of randomized points used to create the jagged lightning-bolt effect.

A straight line between two targets is divided into eight segments. Each intermediate point is moved randomly in a perpendicular direction by as much as 28 pixels. Connecting these randomized points with Line2D produces the irregular appearance of electricity. A bright core line and a wider glow line are displayed together, while additional decorative forks make the attack appear more energetic. The complete effect fades over 0.45 seconds.

Chain Lightning deals 15 damage to each selected enemy, increasing to 23 damage during the final-boss stage because of the shared 50% damage bonus.

10.5 10.6.5 Weapon 5: Gravity Bomb

- The Gravity Bomb is a manually activated area-control weapon assigned to number key 5, and it must be purchased for 50 Weapon XP.
- Pressing key 5 selects the weapon and launches one bomb towards the nearest enemy within 400 pixels.
- The weapon has a 3.4-second cooldown and operates through three states: flying, pulling and exploding.

```

25  ▾ func trigger_attack() -> void:
26  ▾  » if not active or cooldown_left > 0.0:
27    » » return
28    » var target := _find_nearest_enemy_in_range()
29  ▾  » if target != null:
30    » » do_attack(target)
31
32
33  ▾ func _find_nearest_enemy_in_range() -> Node2D:
34    » var nearest: Node2D = null
35    » var nearest_distance := attack_range
36  ▾  » for node in get_tree().get_nodes_in_group("enemy"):
37    » » var enemy := node as Node2D
38  ▾  » » if enemy == null or not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
39    » » » continue
40    » » var distance := global_position.distance_to(enemy.global_position)
41  ▾  » » if distance <= nearest_distance:
42    » » » nearest_distance = distance
43    » » » nearest = enemy
44    » return nearest

```

Manual activation and nearest-enemy selection of the Gravity Bomb.

trigger_attack() is called by the Weapon Manager when the player presses key 5. The function prevents activation when the weapon is inactive or still on cooldown. It then searches for the "enemy" group and selects the nearest living enemy within the 400-pixel attack range. If no enemy is available, the bomb is not launched, and the cooldown is not consumed.

```

47  ▾ func do_attack(target: Node2D = null) -> void:
48  ▾  >|  if cooldown_left > 0.0 or bomb_scene == null:
49      >|  >|  return
50
51      >|  cooldown_left = attack_cooldown
52      >|  var target_position := global_position + Vector2.RIGHT * attack_range
53  ▾  >|  if target != null:
54      >|  >|  target_position = target.global_position
55  ▾  >|  elif player != null and "last_direction" in player:
56      >|  >|  target_position = global_position + player.last_direction.normalized() * a
57
58      >|  var bomb := bomb_scene.instantiate() as Node2D
59  ▾  >|  if bomb == null:
60      >|  >|  return
61      >|  var scene_tree := get_tree()
62      >|  var spawn_parent: Node = scene_tree.current_scene
63  ▾  >|  if spawn_parent == null:
64      >|  >|  spawn_parent = scene_tree.root
65      >|  spawn_parent.add_child(bomb)
66      >|  bomb.call("launch", global_position, target_position, damage, target)

```

Creation and launch configuration of the Gravity Bomb projectile.

After finding a target, the weapon resets its cooldown to 3.4 seconds and creates a new bomb from GravityBombProjectile.tscn. The bomb is added to the current game scene so it can move independently of the player. The launch() call provides the player's position, the target position, the final explosion damage and the target reference. Supplying the target reference allows the projectile to follow a moving enemy instead of travelling only towards its original position.

```

52  ▾ func _physics_process(delta: float) -> void:
53    ▸ | _state_time += delta
54    ▸ | _visual_rotation += delta * (5.0 if state == BombState.PULLING else 2.0)
55
56  ▾ ▸ | match state:
57    ▾ ▸ | ▸ | BombState.FLYING:
58      ▸ | ▸ | ▸ | _update_flying(delta)
59    ▾ ▸ | ▸ | BombState.PULLING:
60      ▸ | ▸ | ▸ | _updatepulling(delta)
61    ▾ ▸ | ▸ | BombState.EXPLODING:
62    ▾ ▸ | ▸ | ▸ | if _state_time >= explosion_duration:
63      ▸ | ▸ | ▸ | ▸ | queue_free()
64
65    ▸ | queue_redraw()
66
67
68  ▾ func _update_flying(delta: float) -> void:
69  ▾ ▸ | if not _update_tracked_target_position():
70    ▸ | ▸ | return
71    ▸ | var current_speed := tracking_speed if is_instance_valid(tracked_target) else
72    ▸ | global_position = global_position.move_toward(target_position, current_speed *
73  ▾ ▸ | if global_position.distance_to(target_position) <= 5.0:
74    ▸ | ▸ | change_state(BombState.PULLING)

```

Gravity Bomb state management and movement towards the tracked target.

The projectile uses a state machine to control its behaviour. In the FLYING state, it continuously updates the enemy's position and moves towards it at a tracking speed of 520 pixels per second. When the bomb comes within 5 pixels of the target position, it changes to the PULLING state. The same physics function later controls the pulling phase and removes the bomb after its explosion animation has finished.

```

77  ▾ func _update_pulling(delta: float) -> void:
78  ▾> | # Keep the active gravity field attached to its original target. If that
79  > | # enemy dies, the last valid position is retained and the bomb still ends.
80  ▾> | if not _update_tracked_target_position():
81  > | > | return
82  ▾> | if is_instance_valid(tracked_target):
83  > | > | global_position = global_position.move_toward(target_position, tracking_speed * delta
84
85  > | _damage_tick_time += delta
86  > | var should_damage := _damage_tick_time >= pull_tick_interval
87  ▾> | if should_damage:
88  > | > | _damage_tick_time = 0.0
89
90  ▾> | for enemy in _get_enemies_in_radius():
91  > | > | var distance := enemy.global_position.distance_to(global_position)
92  > | > | var strength_scale := 1.0 - clampf(distance / pull_radius, 0.0, 1.0)
93  ▾> | > | enemy.global_position = enemy.global_position.move_toward(
94  > | > | > | global_position,
95  > | > | > | pull_strength * (0.35 + strength_scale) * delta
96  > | > | )
97  ▾> | > | if should_damage and enemy.has_method("take_damage"):
98  > | > | > | enemy.take_damage(WEAPON_DAMAGE.calculate(pull_tick_damage, self))
99
100 ▾> | if _state_time >= pull_duration:
101 > | > | change_state(BombState.EXPLODING)

```

Enemy attraction and repeated damage inside the Gravity Bomb field.

During the pulling phase, all living enemies inside the 180-pixel radius are moved towards the centre of the bomb. The pulling strength increases as an enemy gets closer to the centre, producing a concentrated group of targets. The field lasts for 1.6 seconds and deals 5 damage every 0.4 seconds. This repeated damage weakens enemies while gathering them for the final explosion.

```

130  ▾ func change_state(new_state: BombState) -> void:
131  >|    state = new_state
132  >|    _state_time = 0.0
133  >|    _damage_tick_time = 0.0
134  ▾ >|    if state == BombState.EXPLODING:
135  >|    >|        _apply_explosion_damage()
136  >|    state_changed.emit(state)
137  >|    queue_redraw()
138
139
140  ▾ func _apply_explosion_damage() -> void:
141  ▾ >|    for enemy in _get_enemies_in_radius():
142  ▾ >|    >|    if enemy.has_method("take_damage"):
143  >|    >|    >|    enemy.take_damage(WEAPON_DAMAGE.calculate(explosion_damage, self))

```

Transition to the exploding state and application of final area damage.

After the pulling duration finishes, the state changes to EXPLODING. This immediately applies 50 damage to every living enemy still inside the 180-pixel radius. The explosion visual remains for 0.28 seconds before the bomb is removed. During the final-boss stage, the repeated pull damage increases from 5 to 8, while the final explosion increases from 50 to 75.

10.6 10.6.6 Weapon 6: Temporal Echo

- Temporal Echo is a manually activated decoy weapon assigned to number key 6, and it must be purchased for 100 Weapon XP.
- It continuously records the player's previous three seconds of movement and animation, even when another weapon is selected.
- Pressing key 6 creates a ghost that replays the recorded movement, attracts nearby enemies, deals area damage, and has a six-second cooldown.


```

31  ▾ # Recording intentionally uses _process instead of _physics_process. Unlike the
32    # other weapons, it must keep collecting history while another weapon is active.
33  ▾ func _process(delta: float) -> void:|
34    >| cooldown_left = maxf(0.0, cooldown_left - delta)
35  ▾ >| if not _can_record_player():
36    >| >| return
37
38    >| recording_time += delta
39    >| sample_timer += delta
40    >| var interval := maxf(sample_interval, 0.001)
41  ▾ >| if sample_timer < interval:
42    >| >| return
43
44  ▾ >| # One current-position snapshot is enough even after a long frame. Recording
45    >| # several identical positions would add data without improving the replay.
46    >| sample_timer = fmod(sample_timer, interval)
47    >| _record_snapshot()
48    >| _remove_expired_snapshots()

```

Continuous recording of the player's recent movement history.

Temporal Echo uses `_process()` so recording continues even when the weapon is not currently selected. A snapshot is recorded every 0.05 seconds, producing approximately 20 samples per second. Each snapshot stores the player's position, current animation, animation frame, frame progress and facing direction. `_remove_expired_snapshots()` removes records older than three seconds, creating a continuously updated history of the player's most recent actions.

```

121  ▾ func trigger_attack() -> void:
122  ▾  » if not active:
123    » » activation_failed.emit("TEMPORAL ECHO IS NOT ACTIVE")
124    » » return
125  ▾  » if cooldown_left > 0.0:
126    » » activation_failed.emit("TEMPORAL ECHO IS RECHARGING")
127    » » return
128  ▾  » if echo_scene == null:
129    » » activation_failed.emit("TEMPORAL ECHO SCENE IS MISSING")
130    » » return
131  ▾  » if snapshots.size() < 2 or get_recorded_duration() < minimum_recording_time:
132    » » activation_failed.emit("NOT ENOUGH MOVEMENT HISTORY")
133    » » return
134    » » Signal TemporalEchoWeapon.activation_failed(reason: String)
135    » » No description available.
136  ▾  » var echo := ect
137  ▾  » if echo == null:
138    » » activation_failed.emit("TEMPORAL ECHO SCENE IS INVALID")
139    » » return

```

```

140  » » var scene_tree := get_tree()
141  » » var spawn_parent: Node = scene_tree.current_scene
142  ▾  » if spawn_parent == null:
143    » » spawn_parent = scene_tree.root
144    » » spawn_parent.add_child(echo)
145    » » echo.begin_replay(snapshots.duplicate(true), damage_per_tick, player_sprite)
146    » » cooldown_left = attack_cooldown
147    » » echo_created.emit(echo)

```

Validation and creation of a Temporal Echo replay.

When the player presses key 6, the weapon verifies that it is active, the cooldown has finished and enough movement history has been recorded. At least two snapshots covering 0.15 seconds are required. A new Echo is then created and receives a duplicate of the snapshots so that later recordings cannot change the active replay. The player's sprite information is also supplied, allowing the ghost to use the same appearance and animations. Finally, the cooldown is reset to six seconds.

```

62  ▾ func _physics_process(delta: float) -> void:
63  ▾  » if _finishing or replay_snapshots.size() < 2:
64  »   » return
65
66  »   replay_time += delta * maxf(replay_speed, 0.01)
67  »   global_position = _get_interpolated_position(replay_time)
68  »   var visual_index := _segment_index
69  ▾  » if replay_time >= replay_duration:
70  »   »   visual_index = replay_snapshots.size() - 1
71  »   _apply_visual_snapshot(replay_snapshots[visual_index])
72
73  »   _damage_timer += delta
74  »   var interval := maxf(damage_interval, 0.01)
75  ▾  » if _damage_timer >= interval:
76  »   »   _damage_timer = fmod(_damage_timer, interval)
77  »   »   _damage_nearby_enemies()
78
79  ▾  » if replay_time >= replay_duration:
80  »   »   _finish_replay()

```

Movement replay, animation restoration and timed damage processing of the Temporal Echo.

The ghost advances through the recorded timeline at normal speed. `_get_interpolated_position()` calculates a smooth position between consecutive snapshots, preventing the Echo from visibly jumping between recorded points. `_apply_visual_snapshot()` restores the recorded animation frame and direction, making the ghost appear to repeat the player's earlier actions. During the replay, the damage timer activates every 0.5 seconds. Once the complete history has been replayed, the Echo fades away and removes itself.

```

132  ▾ func _damage_nearby_enemies() -> void:
133  ▾ ▸ if _finishing or damage_per_tick <= 0:
134    ▸ ▸ return
135    ▸ var scene_tree := get_tree()
136  ▾ ▸ if scene_tree == null:
137    ▸ ▸ return
138  ▾ ▸ for node in scene_tree.get_nodes_in_group("enemy"):
139    ▸ ▸ var enemy := node as Node2D
140  ▾ ▸ if enemy == null or not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
141    ▸ ▸ continue
142  ▾ ▸ if enemy.global_position.distance_to(global_position) > maxf(damage_radius, 0.0):
143    ▸ ▸ continue
144  ▾ ▸ if enemy.has_method("take_damage"):
145    ▸ ▸ enemy.take_damage(WEAPON_DAMAGE.calculate(damage_per_tick, self))

```

```

148  ▾ func can_attract_enemy(enemy: Node2D) -> bool:
149  ▾ ▸ return not _finishing \
150    ▸ and is_instance_valid(enemy) \
151    ▸ and enemy.global_position.distance_to(global_position) <= maxf(attraction_radius, 0.0)

```

```

154  ▾ func take_damage(_amount: int) -> void:
155  ▾ ▸ if _finishing or not is_instance_valid(ghost_sprite):
156    ▸ ▸ return
157    ▸ hits_absorbed += 1
158  ▾ ▸ if is_instance_valid(_hit_tween):
159    ▸ ▸ _hit_tween.kill()
160    ▸ var normal_alpha := clampf(ghost_opacity, 0.05, 1.0)
161    ▸ ghost_sprite.modulate = Color(1.0, 1.0, 1.0, normal_alpha * 0.4)
162    ▸ _hit_tween = ghost_sprite.create_tween()
163  ▾ ▸ _hit_tween.tween_property(
164    ▸ ▸ ghost_sprite,
165    ▸ ▸ "modulate:a",
166    ▸ ▸ normal_alpha,
167    ▸ ▸ 0.12
168    ▸ )

```

Temporal Echo area damage, enemy attraction and decoy-hit response.

Every 0.5 seconds, the Echo deals 10 damage to all living enemies within a 90-pixel radius. It can also attract enemies from as far as 500 pixels away, encouraging them to attack the ghost instead of the real player. When attacked, the Echo records the absorbed hit and briefly flashes, but it does not lose health

or disappear. It continues acting as a decoy until its recorded movement finishes. During the final-boss stage, its damage per tick increases from 10 to 15.

10.7 10.6.7 Weapon 7: Astral Prism Snare

- The Astral Prism Snare is a manually activated crowd-control weapon assigned to number key 7, and it must be purchased for 150 Weapon XP.
- When activated, an autonomous prism searches for and follows an enemy before creating a damaging energy cage that restrains nearby enemies for four seconds.
- The cage ends with an area explosion, the weapon has an eight-second cooldown, and only one Prism Snare can exist at a time.

```
43  ▾ func trigger_attack() -> void:
44  ▾  ▹ if not active:
45    ▹  ▹ activation_failed.emit("PRISM SNARE IS NOT ACTIVE")
46    ▹  ▹ return
47  ▾  ▹ if cooldown_left > 0.0:
48    ▹  ▹ activation_failed.emit("PRISM SNARE IS RECHARGING")
49    ▹  ▹ return
50  ▾  ▹ if is_instance_valid(active_field):
51    ▹  ▹ activation_failed.emit("A PRISM SNARE IS ALREADY ACTIVE")
52    ▹  ▹ return
53  ▾  ▹ if prism_field_scene == null:
54    ▹  ▹ activation_failed.emit("PRISM SNARE SCENE IS MISSING")
55    ▹  ▹ return
56  ▾  ▹ if not is_instance_valid(player):
57    ▹  ▹ player = _find_player()
58  ▾  ▹ if player == null:
59    ▹  ▹ activation_failed.emit("PLAYER NOT FOUND")
60    ▹  ▹ return
61
62  ▹ var initial_target := _find_nearest_enemy(player.global_position, cast_range)
63  ▹ var field := prism_field_scene.instantiate() as PrismField
```

Activation validation and initial target selection for the Astral Prism Snare.

When the player presses key 7, the weapon verifies that it is active, its cooldown has finished and another Prism Snare is not already operating. It then searches for the nearest enemy within the 620-pixel casting range and creates a new Prism Field. Unlike the Gravity Bomb, the Prism can still be launched when no initial

enemy is found. It travels in the player's facing direction and attempts to acquire a target during flight.

```
114  ▾ # Predictive homing leads moving CharacterBody2D targets. If a target dies or
115     # leaves the tree, the prism searches again instead of disappearing immediately.
116  ▾ func _update_seeking(delta: float) -> void:
117  ▾  ▹ if not COMBAT_TARGET_SELECTOR.is_living_enemy(target):
118  ▹  ▹  _set_target(_find_nearest_enemy(global_position, acquisition_range))
119
120  ▾  ▹ if is_instance_valid(target):
121  ▹  ▹  var target_position := _get_predicted_target_position(target)
122  ▹  ▹  var desired_direction := global_position.direction_to(target_position)
123  ▾  ▹  ▹ if desired_direction != Vector2.ZERO:
124  ▾  ▹  ▹  ▹ velocity = velocity.move_toward(
125  ▹  ▹  ▹  ▹ desired_direction * projectile_speed,
126  ▹  ▹  ▹  ▹ homing_acceleration * delta
127  ▹  ▹  ▹  )
128  ▾  ▹  ▹ if global_position.distance_to(target.global_position) <= capture_distance:
129  ▹  ▹  ▹  _begin_snare(target)
130  ▹  ▹  ▹  return
131  ▾  ▹  else:
132  ▾  ▹  ▹  velocity = velocity.move_toward(
133  ▹  ▹  ▹  velocity.normalized() * projectile_speed,
134  ▹  ▹  ▹  homing_acceleration * 0.25 * delta
135  ▹  ▹  )
```

```
139  ▹  ▹  rotation = velocity.angle()
140  ▹  trail_points.append(global_position)
141  ▾  ▹  if trail_points.size() > 14:
142  ▹  ▹  trail_points.pop_front()
143
144  ▾  ▹  if state_time >= maximum_seek_time:
145  ▹  ▹  _finish_field()
```

Autonomous target acquisition and homing movement of the travelling prism.

While seeking, the prism checks whether its current target is still alive. If the target dies, it searches for another enemy instead of disappearing. The prism predicts a moving enemy's future position and gradually adjusts its velocity using a homing

acceleration of 1050. Its maximum travelling speed is 390 pixels per second. When it comes within 28 pixels of an enemy, `_begin_snare()` creates the cage. If it cannot capture an enemy within three seconds, the field ends automatically.

```
238  ▾ func _update_snaring(delta: float) -> void:
239    ▸ _update_cage_geometry(delta)
240    ▸ var enemies := _get_enemies_in_radius(cage_center, trap_radius)
241    ▾ ▸ if COMBAT_TARGET_SELECTOR.is_living_enemy(target) and target not in enemies:
242      ▸   ▸ enemies.append(target)
243    ▾ ▸ for enemy in enemies:
244      ▸   ▸ _mark_enemy_trapped(enemy)
245    ▾ ▸ for enemy in trapped_enemies.duplicate():
246      ▾ ▸   ▸ if not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
247        ▸     ▸   ▸ trapped_enemies.erase(enemy)
248        ▸     ▸   ▸ continue
249      ▸   ▸ _restrain_enemy(enemy, delta)
250
251    ▸ damage_time -= delta
252    ▾ ▸ if damage_time <= 0.0:
253      ▸   ▸ damage_time = maxf(damage_interval, 0.01)
254      ▸   ▸ _damage_trapped_enemies()
255
256    ▾ ▸ if state_time >= trap_duration:
257      ▸   ▸ _begin_collapse()
```

Enemy capture, restraint and timed damage control inside the Prism cage.

After capturing its target, the prism creates a cage with a radius of 105 pixels. Every living enemy inside this radius is added to `trapped_enemies`. The cage continuously restrains these enemies and applies damage every 0.45 seconds. After operating for four seconds, the field changes from the SNARING state to the COLLAPSING state.

```

271  ▾ func _restrain_enemy(enemy: Node2D, delta: float) -> void:
272  ▾ ▸ if not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
273  ▸ ▸ return
274  ▸ var offset := enemy.global_position - cage_center
275  ▸ var hold_radius := trap_radius * 0.58
276  ▾ ▸ if offset.length() > hold_radius:
277  ▸ ▸ enemy.global_position = cage_center + offset.normalized() * hold_radius
278  ▾ ▸ enemy.global_position = enemy.global_position.move_toward(
279  ▸ ▸ cage_center,
280  ▸ ▸ trap_pull_strength * 0.22 * delta
281  ▸ )
282  ▾ ▸ if enemy is CharacterBody2D:
283  ▸ ▸ enemy.velocity = Vector2.ZERO
284
285
286  ▾ func _damage_trapped_enemies() -> void:
287  ▾ ▸ for enemy in trapped_enemies.duplicate():
288  ▾ ▸ ▸ if not COMBAT_TARGET_SELECTOR.is_living_enemy(enemy):
289  ▸ ▸ ▸ trapped_enemies.erase(enemy)
290  ▸ ▸ ▸ continue
291  ▸ ▸ enemy.take_damage(WEAPON_DAMAGE.calculate(damage_per_tick, self))
292  ▾ ▸ for prism in cage_prisms:
293  ▾ ▸ ▸ if is_instance_valid(prism):
294  ▸ ▸ ▸ prism.flash()

```

Movement restrictions and repeated damage applied to enemies trapped by the Prism.

Enemies attempting to move beyond the cage's inner boundary are placed back inside it. They are also continuously pulled towards the centre, and the velocity of CharacterBody2D enemies is reset to zero. This restricts normal enemy movement without disabling their scripts, meaning enemy animations, attacks and boss timers can continue. Every damage interval, each trapped enemy receives 35 damage.


```

318  ▾ func _begin_collapse() -> void:
319      >| state = FieldState.COLLAPSING
320      >| state_time = 0.0
321      >| field_fill.visible = false
322      ▾ >| for prism in cage_prisms:
323          ▾ >| >| if is_instance_valid(prism):
324              >| >| >| prism.collapse_to(global_position, collapse_duration)
325
326  ▾ func _update_collapsing(delta: float) -> void:
327      ▾ >| for enemy in trapped_enemies:
328          ▾ >| >| if is_instance_valid(enemy):
329              >| >| >| _restrain_enemy(enemy, delta)
330              >| var fade := 1.0 - clampf(state_time / maxf(collapse_duration, 0.01), 0.0, 1.0)
331              >| beam_container.modulate.a = fade
332      ▾ >| if state_time >= collapse_duration:
333          >| >| _explode()

```

```

326  ▾ func _update_collapsing(delta: float) -> void:
327      ▾ >| for enemy in trapped_enemies:
328          ▾ >| >| if is_instance_valid(enemy):
329              >| >| >| _restrain_enemy(enemy, delta)
330              >| var fade := 1.0 - clampf(state_time / maxf(collapse_duration, 0.01), 0.0, 1.0)
331              >| beam_container.modulate.a = fade
332      ▾ >| if state_time >= collapse_duration:
333          >| >| _explode()
334
335  ▾ func _explode() -> void:
336      >| state = FieldState.EXPLODING
337      >| state_time = 0.0
338      >| var final_damage := WEAPON_DAMAGE.calculate(explosion_damage, self)
339      ▾ >| for enemy in _get_enemies_in_radius(cage_center, explosion_radius):
340          >| >| enemy.take_damage(final_damage)
341          >| beam_container.visible = false
342          >| prism_container.visible = false
343          >| _release_trapped_enemies()
344      >| collapse_exploded.emit(cage_center, explosion_radius, final_damage)

```

Prism cage collapse and final area explosion.

When the four-second trapping period ends, the four cage prisms move towards the centre over 0.4 seconds while the energy beams fade. Enemies remain restrained during this collapse. The field then explodes with a larger radius of 145 pixels and deals 60 damage to every enemy inside the blast. Finally, the trapped-enemy metadata is removed so surviving enemies can move normally again.

During the final-boss stage, the repeated cage damage increases from 35 to 53, while the final explosion increases from 60 to 90.

CHAPTER 1111.0 Interactable Objects & GUI Systems

The interactable objects including level exits, HP orbs, gold coins and other game elements that respond to player actions. The GUI system displays key game information, such as health weapon details, minimap data, pause controls, and game status.

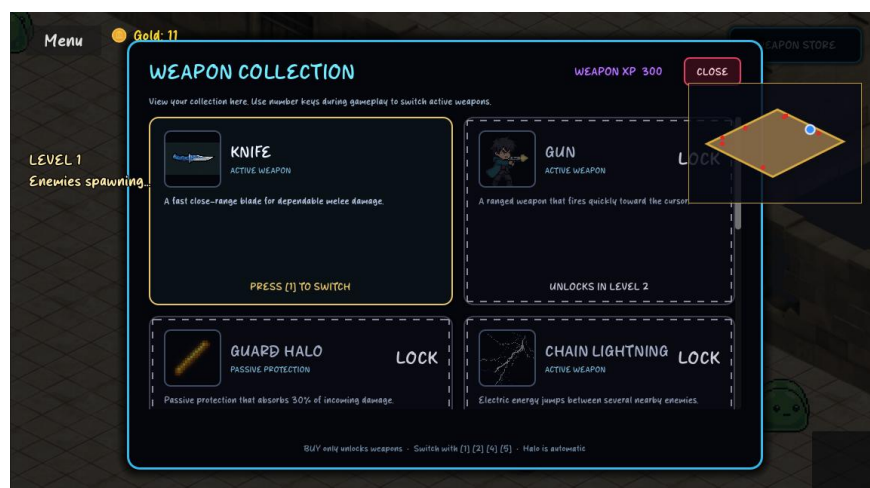
11.1 Interactable Objects Design & Implementation

- Level exit. Each exit has a target level and remains locked until the required conditions are fulfilled. When the system enters the exit area, the system checks whether the exit has been unlocked and whether the object entering the area belongs to the player group. This function is to prevent other enemies or objects from activating the exit. Once activated, the dungeon system can load or activate the next level.
- Gold coins. When the player moves within the specified radius, the coin automatically moves collected by the player and added to the player's coin balance.
- HP Orb. The HP Orb is another object that restores the player's health.
- The game also features different environmental and combat-related objects such as bombs, traps, collapsing bridges, projectiles, and weapons that are linked to battle.

11.2 Game Interface Setup and Implementation (GUIs)

The main gameplay interface contains elements such as:

- Player health. The player has a health bar positioned above the character. The health bar uses a ProgressBar and provides a visual representation of the player's current health. This allows players to immediately determine their remaining health during combat.
- Halo Energy UI. The HaloEnergyUI displays information about the player's Guard Halo ability. The interface tracks current energy, maximum energy, current halo state, energy changes, recharge status, and damage-blocking events.
- Weapon Store UI. The weapon store allows the player to view and unlock weapons using weapon XP. When the store opens, the game pauses to prevent gameplay from continuing when the player interacts with this interface.



11.3 Camera Setup and Implementation

A Camera2D is attached to the player character to provide a third person gameplay view. The camera allows players to remain in the central focus when exploring different areas without losing the gameplay view. This is important because the game has different levels and large environmental areas.

11.4 Canvas and other GUI Setups

The game uses Godot's CanvasLayer to keep the GUI elements fixed on the screen while the camera moves through the game world. The system includes reusable base UI to create common elements such as panels, titles, subtitles, and buttons to maintain consistent interface throughout the game. The pause menu provides options to adjust music and sound effects volume, restart current level, return to main menu and continue the game. The game also includes a dungeon minimap that displays player position and the enemy position. The level-up generates upgrade choices when the player gains a level. Players can choose an upgrade through the interactive card. The game over interface provides options to restart the game or return to the main menu when the players die. The game win interface provides an option to play again or return to the main menu when the player successfully defeats the final boss.

11.5 Game Options and Implementation

The game provides some options that allow the player to manage gameplay and progression.

- **Audio Options.** The pause interface includes settings of control that can be used to adjust game-related settings. Audio is managed through the game's AudioManager that centralizes sound effects and audio playback.
- **Restart and Exit options.** The pause menu allows players to restart the current game or return to the main menu. When restarting, progression such as weapon progress can be handled before reloading the game scene. The main menu also provides the Quit option to exit the game.
- **Upgrade options.** Players can spend gold coins to upgrade, such as increases maximum HP, increases damage or movement speed.
- **Weapon Options.** Players can use weapon XP to unlock additional weapons.

CHAPTER 12

12.0 Project Development Progression

The development of Survive-till-last was divided among team members, with each member responsible for a specific area of the game. Angel Yap Yoon Ying focuses on weapon and inventory systems, Huang Wan Jun focused on the dungeon map, level progression, enemy spawning, traps and minimap. Ng Poh Hui focused on the player controller and level-up system, while Tiu Han Xuen focused on enemy AI, enemy waves, and boss encounter system.

12.1 Project Schedule

The project schedule was structured around the 3 development milestones. Milestone 1 focused on implementing the basic functionality of each assigned system. Angel Yap Yoon Ying developed the basic weapon and projectile system, Huang Wan Jun created the basic 3D dungeon environment and camera positioning, Ng Poh Hui implemented player movement and keyboard controls, and Tiu Han Xuen developed a basic enemy that could track and move towards the player. Milestone 2 expanded the prototype by introducing multiple weapons, inventory slots, dungeon obstacles, enemy spawning and waves, multiple enemy behaviors, experience collection, and level-up systems. Milestone 3 focused on completing and stabilizing all systems, including weapon pooling and inventory completion, additional dungeon levels and bosses, final player stat balancing, stable enemy behaviors, and multi-phase final boss.

CHAPTER 13References

ChillyRoom. (n.d.). *Soul Knight*. <https://soulknight.chillyroom.com/et>

Mixkit. (n.d.). *Download free Sound Effects for Videos – Mixkit*. Mixkit.Co. <https://mixkit.co/free-sound-effects/>

Pixabay. (2024). *Pixabay*. Pixabay.Com. <https://pixabay.com/sound-effects/>

Poncle. (n.d.). *Enemies. Vampire Survivors Wiki*. <https://vampire-survivors.fandom.com/wiki/Enemies>

Boss: Demon Slime by chierit (2021). <https://chierit.itch.io/boss-demon-slime>.